Exact Smooth-Rotation Estimation in Near-Linear Time for ARAP Surface Deformation

Mauricio Cele Lopez Belon

Abstract. As-Rigid-As-Possible deformation with Smooth Rotations (SR-ARAP [6]) has recently gained popularity in applications that need an isometric surface mapping, because the quality of its results is comparable to that of more costly volumetric techniques operating on tetrahedral meshes. SR-ARAP minimizes a non-linear least-squares energy with a local/global optimization. Its local step estimates the rotation of each surface cell together with the rotations of its neighbors, so the rotations are codependent across the whole surface. The local step is currently solved by relaxation, and since only a few relaxation iterations can be afforded per step, the rotations it produces are approximate. We show that, with linearized quaternion rotations, the local step can be solved exactly. After a diagonal scaling it becomes a single sparse, symmetric positive definite linear system, which we prove to be the exact stationarity condition of a well-defined quadratic energy. A feedback term towards the previous rotations bounds the system's smallest eigenvalue independently of the mesh, which keeps it well conditioned. We prove that preconditioning the system with a Cholesky factorization of its position-independent part, which is factored once per set of constraints, bounds the condition number by a constant that shrinks as the mesh is refined. Conjugate gradients then reach the exact rotations, to 10^{-12} rad, in 5 to 20 iterations of linear cost, so the exact local step scales nearly linearly with the mesh and is up to 49 times faster than a direct factorization on large meshes. Compared with the original SR-ARAP local step, the converged shapes have 5–12% lower ARAP distortion and smoother rotations, and they reach the quality at which SR-ARAP stops 2.1 to 4.3 times sooner. Putting the unit-norm constraint back, one cell at a time, shows what the linearization does: it freezes the Lagrange multiplier of the constraint, and the exact multiplier differs from the frozen one by the cell's distortion and the disagreement of its neighbors, which keeps the linearized rotations within 0.15° of the exact ones. The same analysis explains why large twists converge slowly: a handle turned by 180° has a rotation orthogonal, as a quaternion, to the rotations of its neighbors, and its cells stay behind;



(A) Rest (B) Reaching (C) Victory (D) Kick

FIGURE 1. The Armadillo (172,974 vertices) posed with six handles (orange spheres). Rotations are solved in closed form on a 502-vertex control mesh and transferred to the full surface by harmonic interpolation; a complete interactive frame takes 30 ms on a laptop CPU, of which its three exact local steps take 2.6 ms.

letting them take their own rotation in that case reaches the same shapes five times sooner. Combined with a multiresolution scheme that transfers the rotations by harmonic interpolation, the method deforms a 173,000-vertex model interactively, with the exact local steps taking 2.6 ms of a 30 ms frame.

Mathematics Subject Classification (2010). Parallel algorithms 68W10; Clifford algebras, spinors 15A66.

Keywords. Geometric Algebra, Quaternion Estimation, Mesh Deformation.

1. Introduction

High-quality surface deformation aims to preserve the local properties of a shape as much as possible, removing distortion in the form of shear and stretch. As-Rigid-As-Possible deformation with Smooth Rotations (SR-ARAP) [6] creates a surface mapping that minimizes the deviation from rigid behavior at the local scale. It also makes the rigid transformations vary smoothly over local neighborhoods, distributing distortion uniformly over the surface. The resulting deformations preserve the shape not only at small scales but also at large scales, which significantly increases the quality of the final result.

The current SR-ARAP method has two main disadvantages. The first is the local step, which looks for the best rotation of each surface cell while keeping neighboring rotations similar to each other. It needs to converge globally over the whole mesh. Currently the local step is solved with a relaxation

method: the rotation of each cell is optimized while its neighbors' rotations are kept fixed, and the process is repeated until convergence. Although the cost of each relaxation iteration can be reduced considerably by avoiding SVD computations [13], a fixed number of iterations is usually performed in order to bound the solver time. That compromises the quality of the results, since a fixed number of iterations (usually two or three) is not enough to reach optimal rotations across the surface. The second disadvantage is the high computational cost, which increases drastically with the number of vertices: SR-ARAP is a non-linear technique that requires several local/global iterations to converge.

In this paper we address both drawbacks. First, we solve the local step as a single linear system of equations, avoiding the need to solve a series of codependent SVD problems iteratively. We show that, after a diagonal scaling, this system is symmetric positive definite and is the exact stationarity condition of a well-defined quadratic energy (Section 5.4). We bound its condition number and show that a preconditioner built from its position-independent part, factored once per set of constraints, makes conjugate gradients converge in a number of iterations that does not grow with the mesh (Section 5.7); the exact local step therefore costs nearly linear time, and on the meshes we tested it reaches the quality of the original SR-ARAP sooner. Second, we propose a simple multiresolution method: the non-linear problem is first solved on a simplified surface mesh, and the optimized rotations are then transferred to the full-resolution mesh by harmonic interpolation, reusing the Laplacian matrix of the global step.

We also examine the linearization itself. Putting the constraint $|R_i| = 1$ back for one cell at a time turns the cell problem into the minimization of a quadratic on the unit sphere, whose stationarity condition is the linearized cell equation with its Lagrange multiplier left free (Section 5.8). The linearization amounts to freezing that multiplier, and we show that the frozen value is a lower bound of the true one, with a gap made of the cell's distortion and of the disagreement of its neighbors' rotations; we solve the exact cell with Newton's method on the secular equation, with a start that makes it converge monotonically. The exact cells confirm that the linearization is accurate, and they reveal why large twists converge slowly: when a handle is turned by 180°, its rotation is orthogonal to those of its neighbors, and the rule that smooths the handle cells keeps them behind. A simple adaptive rule removes that delay without changing the result (Section 5.9).

2. Related Work

Since the focus of this paper is As-Rigid-As-Possible (ARAP) surface deformation, we only review previous work on ARAP deformation. ARAP surface modeling was introduced by Sorkine and Alexa [12] to produce robust and physically plausible deformations by minimizing the deviation from rigidity of the local transformation associated with every vertex. Although results on

low-resolution meshes look close to physical deformations, the method does not achieve satisfactory results on high-resolution meshes. The reason is that the ARAP energy is minimized by preserving rigidity on large parts of the mesh at the expense of concentrating all the distortion on small parts of it. This problem was solved by the Smooth Rotation enhanced ARAP method (SR-ARAP) of Levi and Gotsman [6], in which the deformation energy is extended with a term that penalizes differing rotations of nearby vertices, significantly increasing the quality of the final result. Chao *et al.* [1] derive the ARAP energy from the elastic energy of continuum mechanics, showing a consistent discretization for volumetric ARAP with tetrahedral cells and for 2D ARAP with triangular cells. However, the ARAP energy does not have a consistent discretization for triangulated surfaces in 3D. SR-ARAP [6] also addresses this issue, providing an energy with a consistent discretization for surfaces in 3D. Further enhancements to SR-ARAP have addressed performance on large meshes [10, 13], made the physical stiffness adjustable [2, 5], and introduced local scaling to the rotations, turning rigid transformations into similarity transformations [4].

3. As-Rigid-As-Possible Shape Deformation with Smooth Rotations

Let Q be the rest mesh, with vertices $q_1, \ldots, q_m$, and P the deformed mesh, with vertices $p_1, \ldots, p_m$ and the same connectivity. Denote by $N(i)$ the set of 1-ring neighbors of vertex i, by $d_i = |N(i)|$ its degree (valence), and by $q_{ij} = q_j - q_i$ and $p_{ij} = p_j - p_i$ the directed edges. The discrete As-Rigid-As-Possible energy with Smooth Rotations (SR-ARAP) is

$$E(P, R) = \sum_{i=1}^{m} \Big(\sum_{j \in N(i)} c_{ij} \, \| R_i \, q_{ij} \, \tilde{R}_i - p_{ij} \|^2 + \alpha A \sum_{j \in N(i)} w_{ij} \, \| R_j - R_i \|^2 \Big), \tag{3.1}$$

where $R_1, \ldots, R_m \in \mathbb{H}$ are unit quaternions, $\tilde{R}_i$ is the conjugate of R_i, and edge vectors are identified with pure quaternions, so that $R_i \, q_{ij} \, \tilde{R}_i$ is the rest edge q_{ij} rotated by R_i. The membrane weights $c_{ij} = \max\big(0, \frac{1}{2}(\cot \gamma_{ij} + \cot \delta_{ij})\big)$ are the familiar cotangent weights, where γ_{ij} and δ_{ij} are the angles opposite the edge ij. They are symmetric, $c_{ij} = c_{ji}$, and non-negative, but not normalized. The bending weights w_{ij} are positive and normalized per vertex, $\sum_{j \in N(i)} w_{ij} = 1$; in this paper $w_{ij} = 1/d_i$. Since d_i varies from vertex to vertex, they are *not* symmetric in general: $w_{ij} \neq w_{ji}$. A is the mesh area, which makes the energy scale invariant, and α is a positive scalar parameter.

The main idea of the method is to break the surface into overlapping cells and keep the transformation of each cell as rigid as possible in the least-squares sense. The cells must overlap to avoid stretching or shearing the surface at cell boundaries. The first term is a membrane energy, which penalizes stretching and shearing of a cell; the second term is a bending energy, which penalizes the difference between a cell's rotation and the rotations of

its neighboring cells. The membrane term lowers the distortion of a cell by keeping the differential of the map close to rigid. The bending term keeps the variation of the rotations within a cell's neighborhood low, so that the neighborhood transforms as a unit as much as possible.

The vertices of Q are in their original positions, the vertices of P are the deformed positions, and the quaternion R_i is the best rigid transformation, in the least-squares sense, relating the original and the deformed cell. This is a non-linear optimization problem, typically solved by an iterative local/global method that solves two linear sub-problems in each iteration. The first, the *local step*, keeps the vertices of P constant and finds the best rotation R_i for each cell. The second, the *global step*, keeps the rotations R_i constant and computes the optimal deformed vertices p_i in the least-squares sense. Each step can only lower the energy, so the iterations decrease it monotonically.

Both steps will be treated with the same four moves, which we spell out once because they recur throughout the paper: (1) write the energy as a function of the unknowns of the step only; (2) set its gradient to zero, which gives a linear system; (3) substitute the values that are fixed by the user's constraints, which leaves a square system in the free unknowns; and (4) show that this system is symmetric positive definite, so that it has a unique solution and can be solved with a Cholesky factorization or by conjugate gradients. Section 4 applies them to the positions, where they are classical. Section 5 applies them to the rotations, where the second and fourth moves need more care, and then shows how the resulting system is solved in near-linear time.

4. Global Step

The global step computes the optimal vertices $p_1, \ldots, p_m$ for fixed rotations:

$$\min_{p_1,\ldots,p_m \in \mathbb{R}^3} \sum_{i=1}^{m} \sum_{j \in N(i)} c_{ij} \, \| R_i \, q_{ij} \, \tilde{R}_i - p_{ij} \|^2 .$$

The bending term of (3.1) does not depend on the positions, so it is omitted. To shorten the notation, write $t^i_{ij} = R_i \, q_{ij} \, \tilde{R}_i$ for the rest edge q_{ij} rotated by the rotation of cell i; it is a known vector in this step.

Where p_i appears. A vertex p_i enters the energy in two kinds of terms: in its own cell, through every edge $p_{ij} = p_j - p_i$ with $j \in N(i)$, and in the cell of each neighbor j, through the edge $p_{ji} = p_i - p_j$. Differentiating one term of each kind,

$$\frac{\partial}{\partial p_i} \| t^i_{ij} - (p_j - p_i) \|^2 = 2 \left(t^i_{ij} - p_{ij} \right),$$
$$\frac{\partial}{\partial p_i} \| t^j_{ji} - (p_i - p_j) \|^2 = -2 \left(t^j_{ji} - p_{ji} \right) = 2 \left(t^j_{ij} - p_{ij} \right),$$

where the last equality uses $q_{ji} = -q_{ij}$, hence $t^j_{ji} = -t^j_{ij}$, and $p_{ji} = -p_{ij}$. Both kinds of terms carry the same weight, since $c_{ij} = c_{ji}$.

Stationarity. Summing over the neighbors and setting the gradient to zero gives $\sum_j c_{ij}\,(t^i_{ij} + t^j_{ij} - 2\,p_{ij}) = 0$, that is, the linear system [7]

$$\sum_{j\in N(i)} c_{ij}\,(p_j - p_i) = \sum_{j\in N(i)} \frac{c_{ij}}{2}\,\big(R_i\,q_{ij}\,\tilde{R}_i + R_j\,q_{ij}\,\tilde{R}_j\big). \tag{4.1}$$

Each edge of the deformed mesh is asked to match the average of the rest edge rotated by the rotations of its two end cells. In matrix form the system reads $\mathrm{L\,P} = \mathrm{B}$. Here L is the $m \times m$ discrete Laplace–Beltrami operator, $(\mathrm{L\,P})_i = \sum_{j\in N(i)} c_{ij}(p_j - p_i)$, P is the $m \times 3$ matrix of target positions, and B is the $m \times 3$ matrix whose ith row is the right-hand side of Equation (4.1). The operator L is symmetric and negative semi-definite, since

$$-x^T \mathrm{L}\,x = \sum_{\{i,j\}\in\mathcal{E}} c_{ij}\,(x_i - x_j)^2 \geq 0 \quad \text{for all } x \in \mathbb{R}^m,$$

where $\mathcal{E}$ is the set of mesh edges, each counted once.

4.1. Positional constraints

Constraints of the form $p_i = p_i^{const}$ for i in a set $\mathcal{C}$ of constrained vertices are incorporated by substituting the corresponding variables. Let $\mathcal{F}$ be the remaining (free) vertices. The equations of the constrained vertices are dropped, since their positions are not unknowns any more. Partitioning L, P and B accordingly, the rows of $\mathcal{F}$ read $\mathrm{L}_{\mathcal{F}\mathcal{F}}\mathrm{P}_{\mathcal{F}} + \mathrm{L}_{\mathcal{F}\mathcal{C}}\mathrm{P}_{\mathcal{C}} = \mathrm{B}_{\mathcal{F}}$. Moving the known part to the right and changing signs, so that the matrix is positive rather than negative,

$$(-\mathrm{L}_{\mathcal{F}\mathcal{F}})\,\mathrm{P}_{\mathcal{F}} = -\mathrm{B}_{\mathcal{F}} + \mathrm{L}_{\mathcal{F}\mathcal{C}}\,\mathrm{P}^{const}_{\mathcal{C}}. \tag{4.2}$$

This system is square, with one equation per free unknown, so no least-squares formulation (normal equations $\mathrm{L}^T\mathrm{L}$) is needed; forming the normal equations would square the condition number and increase the fill-in of the factorization.

Proposition 4.1. *If every connected component of the mesh, connected through edges with $c_{ij} > 0$, contains a constrained vertex, then $-\mathrm{L}_{\mathcal{F}\mathcal{F}}$ is symmetric positive definite.*

Proof. $-\mathrm{L}_{\mathcal{F}\mathcal{F}}$ is a principal submatrix of the symmetric matrix $-\mathrm{L}$, so it is symmetric. For $x \in \mathbb{R}^{|\mathcal{F}|}$, extend it by zero on $\mathcal{C}$; then $x^T(-\mathrm{L}_{\mathcal{F}\mathcal{F}})x = \sum_{\{i,j\}\in\mathcal{E}} c_{ij}(x_i - x_j)^2 \geq 0$. Equality holds only if x is constant on every component, and since every component contains a constrained vertex, where the extended x is zero, this forces $x = 0$. □

Equation (4.2) is therefore solved with a sparse Cholesky factorization of $-\mathrm{L}_{\mathcal{F}\mathcal{F}}$. This matrix depends only on the rest mesh and the constraint set, not on the rotations, so it is factored once; every global step then costs two sparse triangular solves per coordinate, and the three coordinates share the factorization. The local step will not be this fortunate: its matrix changes with the positions, and Section 5.7 shows how to recover a factor-once method for it.

5. Local Step

In the local step the positions are fixed and the rotations are the unknowns. Unlike the positions, the rotations cannot be found one cell at a time: the bending term couples the rotation of every cell to those of its neighbors, and through them to the whole mesh. SR-ARAP handles this coupling by relaxation. It repeatedly visits the cells and finds the best rotation of each one with its neighbors' rotations held fixed, and since only two or three such sweeps can be afforded per iteration, the rotations it returns are approximate.

We follow the four moves of Section 3 instead. First we make the energy quadratic by treating quaternions as vectors of $\mathbb{R}^4$ (Section 5.1). We then write the equation of a single cell, which is the equation relaxation solves (Section 5.2), and require it to hold at every cell at once, which gives one linear system for all the rotations (Section 5.3). After a scaling of its rows the system is symmetric, and we identify the energy it minimizes (Section 5.4). Substituting the constrained rotations, we prove that it is positive definite (Section 5.5), discuss the role of the feedback term (Section 5.6), and finally show how it is solved in near-linear time (Section 5.7).

5.1. Rotations as vectors of $\mathbb{R}^4$

Three observations turn the membrane term of a cell into a quadratic form in the coefficients of its quaternion.

(i) The conjugate can be removed. Multiplying a quaternion by a unit quaternion does not change its norm. Multiplying $R_i\, q_{ij}\, \tilde{R}_i - p_{ij}$ on the right by R_i and using $\tilde{R}_i R_i = 1$,

$$\|R_i\, q_{ij}\, \tilde{R}_i - p_{ij}\| = \|(R_i\, q_{ij}\, \tilde{R}_i - p_{ij})\, R_i\| = \|R_i\, q_{ij} - p_{ij}\, R_i\|.$$

(ii) The result is linear in R_i. Both $R_i\, q_{ij}$ and $p_{ij}\, R_i$ are linear in the four coefficients of R_i, so $R_i\, q_{ij} - p_{ij}\, R_i = M_{ij} R_i$ for a 4×4 real matrix M_{ij} that depends only on q_{ij} and p_{ij}. Section 7 gives it explicitly.

(iii) The membrane term is a quadratic form. Since $\|M_{ij} R_i\|^2 = R_i^T M_{ij}^T M_{ij} R_i$, summing over the cell gives

$$\sum_{j \in N(i)} c_{ij}\, \|R_i\, q_{ij} - p_{ij}\, R_i\|^2 = R_i^T M_i\, R_i, \qquad M_i = \sum_{j \in N(i)} c_{ij}\, M_{ij}^T M_{ij}. \quad (5.1)$$

Each $M_{ij}^T M_{ij}$ is symmetric positive semi-definite and the weights c_{ij} are non-negative, so M_i is a symmetric positive semi-definite 4×4 matrix.

The bending term needs no work: $\|R_j - R_i\|^2$ is already the squared Euclidean distance of $\mathbb{R}^4$.

Linearization. Observation (i) holds only for $|R_i| = 1$. We now drop this constraint and treat the R_i as arbitrary vectors of $\mathbb{R}^4$. All energies of the local step then become quadratic, and their stationarity conditions linear; the quaternions are normalized after the solve. As shown in [13], this linearization gives an accurate approximation of the optimal quaternion in the least-squares sense, and our results confirm that it does not affect the accuracy of SR-ARAP rotations in any significant way. Two things deserve a

comment. First, without the unit norm nothing in the membrane term prevents the trivial answer $R_i = 0$. What keeps the rotations away from zero are the known rotations that will appear on the right-hand side of the equations: those of the constrained cells (Section 5.5) and the previous rotations of the feedback term (Section 5.2). Second, R and $-R$ represent the same rotation but are far apart in $\mathbb{R}^4$, so the bending term is meaningful only if neighboring quaternions lie in the same hemisphere; starting every local step from the previous rotations keeps them there.

The linearized SR-ARAP energy of the local step is then, with the weights $w_{ij} = 1/d_i$ of Section 3,

$$E(R) = \sum_{i=1}^{m} \Big(R_i^T M_i \, R_i + \alpha A \sum_{j \in N(i)} w_{ij} \, \|R_j - R_i\|^2 \Big). \tag{5.2}$$

5.2. The equation of one cell

We start with the simplest problem: the best rotation of a single cell i when the rotations R_j of its neighbors are known. It is the problem relaxation solves for one cell at a time, and its equation is the building block of everything that follows.

Feedback. Before writing the equation we add one term to the energy of the cell. During interactive editing only a few local/global iterations run per frame, and the rotations of one frame should stay close to those of the previous frame; otherwise they can jump between frames and, for large rotations, even reverse their sense of rotation, which shows as artifacts in the animation. We therefore let the previous rotation R_i^{prev} of the cell act as one more neighbor, with weight $e > 0$ where each true neighbor has weight 1. The energy of the cell becomes

$$\begin{gathered} E_i(R_i) = R_i^T M_i \, R_i + \alpha A \Big(\sum_{j \in N(i)} w_{ij} \, \|R_i - R_j\|^2 + \epsilon_i \, \|R_i - R_i^{prev}\|^2 \Big), \\ w_{ij} = \frac{1}{d_i + e}, \quad \epsilon_i = \frac{e}{d_i + e}. \end{gathered} \tag{5.3}$$

The bending weights are renormalized to make room for the new neighbor: they still sum to one, $\sum_{j \in N(i)} w_{ij} + \epsilon_i = 1$, like those of SR-ARAP, and for $e = 0$ they are exactly SR-ARAP's, $w_{ij} = 1/d_i$. From now on w_{ij} denotes these renormalized weights. Section 5.6 discusses what the feedback brings and what it costs.

The gradient. Two rules suffice: for a symmetric matrix M, the gradient of $R^T M R$ is $2MR$; and for a fixed vector X, the gradient of $\|R - X\|^2$ is $2(R - X)$. Applying them term by term,

$$\tfrac{1}{2} \frac{\partial E_i}{\partial R_i} = M_i \, R_i + \alpha A \sum_{j \in N(i)} w_{ij} \, (R_i - R_j) + \alpha A \, \epsilon_i \, (R_i - R_i^{prev}).$$

The terms that contain R_i add up to $\big(M_i + \alpha A \, (\sum_j w_{ij} + \epsilon_i) \, I\big) R_i = (M_i + \alpha A \, I) \, R_i$, thanks to the normalization of the weights.

The cell equation. Setting the gradient to zero, the best rotation of cell i satisfies

$$\begin{aligned}(M_i + \alpha A\, I)\, R_i &= \alpha A \sum_{j \in N(i)} w_{ij}\, R_j + \alpha A\, \epsilon_i\, R_i^{prev} \\ &= \frac{\alpha A}{d_i + e}\Big(\sum_{j \in N(i)} R_j + e\, R_i^{prev} \Big).\end{aligned} \tag{5.4}$$

The right-hand side is a weighted average of the neighbors' rotations and the previous rotation, scaled by αA; the matrix on the left adds the membrane term of the cell, which pulls the average towards the rotation that best fits the cell's own edges. Since $M_i \succeq 0$, the matrix $M_i + \alpha A\, I \succeq \alpha A\, I$ is positive definite, so for given neighbors (5.4) has a unique solution, found with one 4×4 linear solve. Relaxation sweeps through the cells and solves (5.4) for each one with the current rotations of its neighbors: it is a block Gauss–Seidel iteration, and after a few sweeps its rotations satisfy (5.4) only approximately.

5.3. All cells at once

Instead of solving the cell equations one at a time, we now ask for rotations that satisfy (5.4) at every cell simultaneously; these are the rotations relaxation converges to. Every cell equation is linear in the rotations, so together they form one linear system, with four equations and four unknowns per cell. For the moment we treat every cell as unknown; the constrained cells are handled in Section 5.5.

The system is not symmetric. Moving the neighbors to the left, the equations of cell i read

$$(M_i + \alpha A\, I)\, R_i - \frac{\alpha A}{d_i + e} \sum_{j \in N(i)} R_j = \alpha A\, \epsilon_i\, R_i^{prev}.$$

The coefficient of R_j in the equations of cell i is $-\frac{\alpha A}{d_i + e}\, I$, while the coefficient of R_i in the equations of cell j is $-\frac{\alpha A}{d_j + e}\, I$. The two differ whenever $d_i \neq d_j$, which happens on every mesh that is not perfectly regular, so the system is not symmetric.

Why symmetry matters. A symmetric positive definite system can be solved with a Cholesky factorization, which needs no pivoting and half the work of an LU factorization, or by conjugate gradients, which only need products with the matrix. Moreover, a symmetric system is the stationarity condition of an energy (Section 5.4), which tells us what its solution minimizes. None of this is available for a non-symmetric system.

Restoring symmetry. Multiplying the equations of a cell by a nonzero number does not change the solution. Multiplying those of cell i by

$$s_i = \frac{d_i + e}{\alpha A}$$

turns every coupling coefficient into $-I$:

$$s_i\, (M_i + \alpha A\, I)\, R_i - \sum_{j \in N(i)} R_j = e\, R_i^{prev}. \tag{5.5}$$

In matrix form this is $\mathrm{K}\,\mathrm{R} = e\,\mathrm{R}^{prev}$, where R and R^{prev} are the $4m \times 1$ column vectors stacking the coefficients of the quaternions R_i and R_i^{prev}, and K is the $4m \times 4m$ matrix with 4×4 blocks

$$\mathrm{K}_{ii} = s_i\,(M_i + \alpha A\,I), \qquad \mathrm{K}_{ij} = -I \text{ if } \{i,j\} \in \mathcal{E}, \qquad \mathrm{K}_{ij} = 0 \text{ otherwise.}$$

K is symmetric, because the adjacency of the mesh is.

5.4. The energy minimized by the system

A symmetric linear system always comes from an energy. If K is symmetric, the gradient of $f(x) = \frac{1}{2}x^T\mathrm{K}x - b^Tx$ is $\mathrm{K}x - b$, so $\mathrm{K}x = b$ states that x is a stationary point of f; if K is moreover positive definite, f is strictly convex and its stationary point is its unique minimizer. The following proposition writes this energy for the rotation system in terms of the mesh.

Proposition 5.1. *The system* $\mathrm{K}\,\mathrm{R} = e\,\mathrm{R}^{prev}$ *is the exact stationarity condition of the strictly convex quadratic energy*

$$\tilde{E}(R) = \sum_{i=1}^{m}(d_i+e)\,R_i^T M_i\,R_i + \alpha A \sum_{\{i,j\}\in\mathcal{E}} \|R_i - R_j\|^2 + \alpha A\,e \sum_{i=1}^{m} \|R_i - R_i^{prev}\|^2, \tag{5.6}$$

where $\mathcal{E}$ *is the set of mesh edges, each counted once. Its unique minimizer is therefore the set of rotations that satisfies the cell equation* (5.4) *at every cell simultaneously.*

Proof. We compute the gradient of $\tilde{E}$ with respect to R_i term by term. The membrane term contributes $2(d_i + e)M_iR_i$. The edge $\{i,j\}$ appears once in the bending sum and contributes $2\alpha A(R_i - R_j)$; vertex i has d_i such edges. The feedback term contributes $2\alpha Ae(R_i - R_i^{prev})$. Hence

$$\begin{aligned}
\tfrac{1}{2}\frac{\partial \tilde{E}}{\partial R_i} &= (d_i+e)\,M_i\,R_i + \alpha A \sum_{j\in N(i)} (R_i - R_j) + \alpha A\,e\,(R_i - R_i^{prev}) \\
&= (d_i+e)\,M_i\,R_i + \alpha A\,(d_i+e)\,R_i - \alpha A \sum_{j\in N(i)} R_j - \alpha A\,e\,R_i^{prev} \\
&= \alpha A\Big(s_i\,(M_i + \alpha A\,I)\,R_i - \sum_{j\in N(i)} R_j - e\,R_i^{prev}\Big),
\end{aligned}$$

which vanishes exactly when (5.5) holds. The first two terms of $\tilde{E}$ are positive semi-definite quadratic forms and the feedback term is positive definite, so $\tilde{E}$ is strictly convex and its stationary point is its unique minimizer. □

$\tilde{E}$ is the linearized SR-ARAP energy (5.2) with three differences: the membrane term of cell i is weighted by $d_i + e$, the bending term is the symmetric, unweighted sum over edges, and the feedback appears as an explicit proximal term towards the previous rotations. Solving $\mathrm{K}\,\mathrm{R} = e\,\mathrm{R}^{prev}$ is thus not an approximation of relaxation: it gives in one step the exact minimizer of $\tilde{E}$, which relaxation can only approach iteratively. We call this solution the *closed form* of the local step.

Remark 5.2. Why does the system not minimize the sum of the cell energies, (5.2)? Each cell equation minimizes E_i over R_i with the neighbors held fixed, but in the total energy R_i also appears in the bending terms of its neighbors' cells, so

$$\tfrac{1}{2}\,\frac{\partial E}{\partial R_i} = M_i\,R_i + \alpha A \sum_{j\in N(i)} (w_{ij}+w_{ji})\,(R_i - R_j). \tag{5.7}$$

The cell equations ignore the second weight, w_{ji}, and the two conditions coincide only when $w_{ij} = w_{ji}$, which per-vertex normalized weights do not satisfy. Minimizing E itself would lead to another symmetric system, with the weights $w_{ij} + w_{ji}$. We keep (5.5) because its equations are the cell equations themselves: every rotation is the best one for its own cell given its neighbors, as in the cell-by-cell formulation, and relaxation of (5.4) converges to the same solution (Section 5.5), which lets us measure the error of relaxation exactly (Section 9).

5.5. Constrained rotations and positive definiteness

The vertices inside the handles are constrained, $i \in \mathcal{C}$. Their positions are prescribed, and their rotations are not unknowns of the local step either: each is computed directly from the cell equation without feedback ($e = 0$), with the neighboring rotations R_j^{prev} of the previous iteration,

$$R_i^{const} = (M_i + \alpha A\,I)^{-1}\,\frac{\alpha A}{d_i} \sum_{j\in N(i)} R_j^{prev}, \qquad i \in \mathcal{C}, \tag{5.8}$$

and normalized. Each constrained rotation therefore costs one 4×4 linear solve, which is always possible since $M_i + \alpha A\,I$ is positive definite. This rule has one blind spot: when a handle is turned by a large angle at once, (5.8) detaches the cells of the handle from the handle. Section 5.9 explains why and replaces (5.8), in that situation only, by the cell's own fit; the resulting *adaptive rule* (5.17) is the one our method uses. For what follows it only matters that every R_i^{const} is known before the rotation system is solved.

The reduced system. As in the global step, we keep only the equations of the free cells $\mathcal{F}$ and move the known rotations to the right: $\mathrm{K}_{\mathcal{F}\mathcal{F}}\,\mathrm{R}_{\mathcal{F}} = e\,\mathrm{R}_{\mathcal{F}}^{prev} - \mathrm{K}_{\mathcal{F}\mathcal{C}}\,\mathrm{R}_{\mathcal{C}}^{const}$. Since the coupling blocks of K are $-I$, the four equations of a free cell i read

$$s_i\,(M_i + \alpha A\,I)\,R_i - \sum_{j\in N(i)\cap\mathcal{F}} R_j = e\,R_i^{prev} + \sum_{j\in N(i)\cap\mathcal{C}} R_j^{const}, \qquad i \in \mathcal{F}: \tag{5.9}$$

a constrained neighbor simply adds its rotation to the right-hand side. Equivalently, $\mathrm{R}_{\mathcal{F}}$ minimizes $\tilde{E}$ over the free rotations, with the constrained ones fixed to R_i^{const}.

Proposition 5.3. *$\mathrm{K}_{\mathcal{F}\mathcal{F}}$ is symmetric positive definite, with smallest eigenvalue at least e.*

Proof. It is a principal submatrix of the symmetric matrix K, so it is symmetric. Since M_i is positive semi-definite, the diagonal blocks satisfy $s_i(M_i + \alpha A\,I) \succeq s_i\,\alpha A\,I = (d_i + e)\,I$. For $x \in \mathbb{R}^{4|\mathcal{F}|}$ with blocks x_i, each edge between

two free vertices contributes $-2\,x_i \cdot x_j \geq -\|x_i\|^2 - \|x_j\|^2$ to $x^T \mathrm{K}_{\mathcal{F}\mathcal{F}}\, x$, so vertex i loses at most $\|x_i\|^2$ per free neighbor, and it has at most d_i of them:

$$x^T \mathrm{K}_{\mathcal{F}\mathcal{F}}\, x \geq \sum_{i\in\mathcal{F}} (d_i + e)\,\|x_i\|^2 - \sum_{i\in\mathcal{F}} |N(i)\cap\mathcal{F}|\,\|x_i\|^2 \geq e\,\|x\|^2. \qquad \square$$

Three consequences follow. First, unlike Proposition 4.1, the result holds for any constraint set, even an empty one: the feedback term alone makes the system positive definite. Second, relaxation converges: it is block Gauss–Seidel on (5.9), and Gauss–Seidel converges from any starting point for every symmetric positive definite matrix, a classical result of numerical linear algebra, so the closed form and relaxation share the same solution. Third, (5.9) can be solved with a sparse Cholesky factorization, with no least-squares formulation. Its sparsity pattern is that of the mesh graph with 4×4 blocks and depends only on the mesh and the constraint set, so the symbolic analysis (the fill-reducing ordering) is computed once. The values, however, change in every iteration, since M_i depends on the deformed positions, so a direct solve costs one numeric factorization per local step, whose cost grows as $n^{1.5}$ on surface meshes. Section 5.7 removes that factorization. In either case the quaternions R_i are normalized after the solve.

5.6. The role of the feedback

With the system and its properties in hand, we can now say precisely what the feedback term of Section 5.2 contributes. It has four benefits. (i) *Temporal coherence*: during interactive editing only a few iterations run per frame, and the feedback keeps every local step close to the previous one, so rotations do not pop between frames. (ii) *Large rotations*: near and beyond 180° the linearized fit has two nearly equivalent senses of rotation; anchoring each cell to its previous rotation favors the branch it was following, which prevents abrupt changes in the sense of rotation and lets a large twist spread evenly along the surface as it grows (Section 9.6). (iii) *Conditioning*: it bounds the smallest eigenvalue of the rotation system by e for any mesh and any constraint set (Proposition 5.3), so the system stays solvable even without constraints, and iterative solvers of it converge in a number of iterations independent of the mesh (Section 9). (iv) *Little bias*: it barely changes the converged shape (Remark 5.4). Its cost is a damping of the outer local/global loop, which needs more iterations to reach its fixed point when the rotations are large.

Remark 5.4. How much does the feedback change the answer? At a fixed point of the local/global loop the rotations no longer change, $R_i = R_i^{prev}$, so (5.4) becomes $(M_i + \alpha A\, I)R_i = \frac{\alpha A}{d_i+e}\big(\sum_j R_j + eR_i\big)$. Moving the term eR_i to the left gives $M_i R_i + \frac{\alpha A\, d_i}{d_i+e} R_i = \frac{\alpha A}{d_i+e}\sum_j R_j$, and multiplying by $(d_i+e)/d_i$,

$$\Big(\frac{d_i+e}{d_i}\, M_i + \alpha A\, I\Big) R_i = \frac{\alpha A}{d_i} \sum_{j\in N(i)} R_j.$$

This is the cell equation without feedback, (5.4) with $e = 0$, except that the membrane term of each cell is weighted by $(d_i+e)/d_i$, about 1.03 for $e = 0.2$

and a typical valence of 6. The feedback therefore acts as a proximal damping of the iterations rather than as a change of the model. On the bar bent into a 90° arc, the shapes converged with $e = 0.2$ and with $e = 0$ (3,000 iterations each) differ by at most 0.33% of the bar's length, and their ARAP distortions by 0.9%.

5.7. Solving the rotation system in near-linear time

The cost of a direct solve. A sparse Cholesky factorization of $\mathrm{K}_{\mathcal{F}\mathcal{F}}$ creates fill-in: with the best orderings, nested dissection, a surface mesh of n vertices produces $O(n \log n)$ nonzeros in the factor and costs $O(n^{1.5})$ operations to factor. The global step pays this price once, because its matrix does not change. The rotation system pays it in every local step, because its diagonal blocks contain the matrices M_i, which depend on the deformed positions. The remedy is to separate the part of $\mathrm{K}_{\mathcal{F}\mathcal{F}}$ that changes from the part that does not.

Splitting the matrix. Write

$$\mathrm{K}_{\mathcal{F}\mathcal{F}} = \mathrm{C} \otimes I_4 + \mathrm{D}, \qquad \mathrm{D} = \operatorname{diag}\big(s_i M_i\big)_{i \in \mathcal{F}}, \tag{5.10}$$

where C is the $|\mathcal{F}| \times |\mathcal{F}|$ scalar matrix with diagonal $s_i(\alpha A + \eta)$ and the entry -1 for every edge between free vertices, and η is the small diagonal regularization of Section 8. The Kronecker product $\mathrm{C} \otimes I_4$ is the matrix that applies C to each of the four quaternion components separately: arranging the rotations as an $|\mathcal{F}| \times 4$ matrix, one column per component, it multiplies that matrix by C. C depends only on the mesh, the constraint set and the feedback weight. D is block diagonal and positive semi-definite, and carries all the dependence on the deformed positions.

Conjugate gradients. For a symmetric positive definite system, the conjugate gradient method needs only products with the matrix, and its error, measured in the energy norm, decreases at least by the factor $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ per iteration, where κ is the condition number of the matrix. The preconditioned method (PCG) uses a second symmetric positive definite matrix P, the preconditioner, and behaves like conjugate gradients on $\mathrm{P}^{-1}\mathrm{K}_{\mathcal{F}\mathcal{F}}$; each iteration then also solves one system with P. A good preconditioner is close to the matrix, so that $\kappa(\mathrm{P}^{-1}\mathrm{K}_{\mathcal{F}\mathcal{F}})$ is small, and cheap to solve with. The constant part $\mathrm{P} = \mathrm{C} \otimes I_4$ is a natural candidate on both counts: it is all of $\mathrm{K}_{\mathcal{F}\mathcal{F}}$ except the positive semi-definite part D, and since it does not change, it can be factored once, like the Laplacian of the global step. The following proposition shows how good it is.

Proposition 5.5. *Let* $\mu = \max_{i \in \mathcal{F}} s_i\, \lambda_{\max}(M_i)$ *and* $d_{\max} = \max_i d_i$. *Then*

$$\kappa\big(\mathrm{K}_{\mathcal{F}\mathcal{F}}\big) \le \frac{2d_{\max} + e + \mu}{e}\,(1 + \eta/\alpha A), \qquad \kappa\big((\mathrm{C} \otimes I_4)^{-1}\mathrm{K}_{\mathcal{F}\mathcal{F}}\big) \le 1 + \frac{\mu}{e}.$$

Proof. The diagonal of C is $s_i(\alpha A + \eta) = (d_i + e)(1 + \eta/\alpha A) \ge d_i + e$ and its off-diagonal entries in row i sum in absolute value to at most d_i, so by Gershgorin's theorem the eigenvalues of C, and hence those of $\mathrm{C} \otimes I_4$, lie in $[e, (2d_{\max}+e)(1+\eta/\alpha A)]$. The diagonal blocks of D are $s_i M_i$, with eigenvalues

between 0 and μ, so $0 \preceq \mathrm{D} \preceq \mu I$. Adding the two, the eigenvalues of $\mathrm{K}_{\mathcal{FF}}$ lie in $[e, (2d_{\max}+e)(1+\eta/\alpha A)+\mu]$, and the ratio of the endpoints is bounded by the first expression. For the second bound, the eigenvalues of $(\mathrm{C}\otimes I_4)^{-1}\mathrm{K}_{\mathcal{FF}}$ are the values of the generalized Rayleigh quotient

$$\frac{x^T \mathrm{K}_{\mathcal{FF}}\, x}{x^T(\mathrm{C}\otimes I_4)\, x} = 1 + \frac{x^T \mathrm{D}\, x}{x^T(\mathrm{C}\otimes I_4)\, x} \in \Big[1,\, 1+\frac{\mu}{e}\Big],$$

since $0 \le x^T \mathrm{D}\, x \le \mu\|x\|^2$ and $x^T(\mathrm{C}\otimes I_4)\, x \ge e\|x\|^2$. □

What the bounds say. Both bounds are independent of the resolution of the mesh. Moreover μ decreases under refinement: $\lambda_{\max}(M_i) \le \operatorname{tr} M_i = 4\sum_j c_{ij}(\|p_{ij}\|^2 + \|q_{ij}\|^2)$ (Section 7) scales with the squared edge length, while s_i does not depend on the resolution, so for a uniform refinement of a fixed surface with bounded stretching $\mu = O(1/n)$, and the preconditioned condition number tends to 1. We therefore solve (5.9) by PCG with preconditioner $\mathrm{C}\otimes I_4$, warm-started from the previous rotations. By the convergence rate above, the number of iterations needed for a given accuracy is bounded independently of the mesh, and decreases as it is refined.

Cost. C is factored once per constraint set with a sparse Cholesky factorization, like the Poisson systems of Sections 4 and 6.1. Every PCG iteration then costs one product with $\mathrm{K}_{\mathcal{FF}}$, which is $O(n)$, and one application of the preconditioner: two triangular solves with four right-hand sides, $O(n \log n)$ for the nested-dissection orderings of surface meshes. A local step therefore costs $O(n \log n)$, instead of the $O(n^{1.5})$ of a numeric factorization. We stop PCG at a relative residual of 10^{-12}, where its rotations match those of the direct solve to within 10^{-12} rad, so the method keeps the exactness of the closed form. Without a preconditioner, conjugate gradients also converges in a mesh-independent number of iterations by the first bound of Proposition 5.5, at $O(n)$ per iteration: a linear-time solver, at the price of more iterations.

5.8. The linearization seen from the unit sphere

In Section 5.1 we dropped the constraint $|R_i| = 1$, solved, and normalized afterwards. The results confirm that this works, but they do not say *why*, nor where it might not. To find out, we now put the constraint back, for one cell at a time, and solve the cell problem exactly on the unit sphere. Comparing the exact cell with the linearized one will show precisely what the linearization changes, how large the change is, and in which situation it is not small.

The cell energy on the sphere. Take the energy (5.3) of cell i with its neighbors held fixed, and now require $|R_i| = 1$. On the sphere the bending terms simplify: for unit quaternions,

$$\|R_i - R_j\|^2 = |R_i|^2 + |R_j|^2 - 2\, R_i \cdot R_j = 2 - 2\, R_i \cdot R_j,$$

so each of them is a constant minus a term that is *linear* in R_i. Collecting the linear terms, and recalling that the weights w_{ij} and ϵ_i sum to one, the

cell energy becomes, up to a constant,

$$E_i(R_i) = R_i^T M_i\, R_i - 2\, h_i^T R_i,$$
$$h_i = \alpha A\, g_i, \qquad g_i = \sum_{j \in N(i)} w_{ij}\, R_j + \epsilon_i\, R_i^{prev}. \tag{5.11}$$

The vector g_i is a weighted average of unit quaternions with weights that sum to one: the *mean rotation* of the cell's neighborhood, which is a unit vector only when all the neighbors agree. The vector h_i, the *pull* of the neighbors, is that mean scaled by the stiffness αA. On the sphere, the cell looks for the rotation that fits its own edges, the quadratic term, and that points along the pull, the linear term.

The stationarity condition. To minimize (5.11) on the sphere we use a Lagrange multiplier λ for the constraint $|R_i|^2 = 1$. The gradient of $E_i - \lambda(|R_i|^2 - 1)$ vanishes when

$$(M_i - \lambda I)\, R_i = h_i, \qquad |R_i| = 1. \tag{5.12}$$

Geometrically, λ measures the part of the energy gradient that is normal to the sphere: at the solution, $\frac{1}{2}\nabla E_i = M_i R_i - h_i = \lambda R_i$ points along R_i, so it cannot be lowered by moving on the sphere. Now compare (5.12) with the linearized cell equation (5.4), $(M_i + \alpha A\, I) R_i = h_i$. They are the same equation, with the multiplier *frozen* at $\lambda = -\alpha A$. This is what the linearization does, and all it does: instead of choosing λ so that the solution has unit length, it uses a fixed value and normalizes afterwards.

How far is the frozen value from the true one? Multiplying (5.12) by R_i^T and using $|R_i| = 1$ gives the multiplier of a solution R_i^* explicitly, $\lambda^* = R_i^{*T} M_i R_i^* - h_i \cdot R_i^*$. The two terms can be read off the mesh:

Proposition 5.6. *The multiplier of a solution of* (5.12) *satisfies*

$$\lambda^* + \alpha A = R_i^{*T} M_i\, R_i^* + \frac{\alpha A}{2}\, |g_i - R_i^*|^2 + \frac{\alpha A}{2}\Big(\sum_{j \in N(i)} w_{ij}\, |R_j - g_i|^2 + \epsilon_i\, |R_i^{prev} - g_i|^2 \Big), \tag{5.13}$$

a sum of three non-negative terms. In particular $\lambda^* \geq -\alpha A$: the frozen multiplier is a lower bound of the true one.

Proof. From $\lambda^* = R_i^{*T} M_i R_i^* - \alpha A\, g_i \cdot R_i^*$ we get $\lambda^* + \alpha A = R_i^{*T} M_i R_i^* + \alpha A\,(1 - g_i \cdot R_i^*)$. Since $|R_i^*| = 1$, $1 - g_i \cdot R_i^* = \frac{1}{2}|g_i - R_i^*|^2 + \frac{1}{2}(1 - |g_i|^2)$. Finally, for unit vectors X_k averaged with weights $\omega_k \geq 0$ that sum to one, $\sum_k \omega_k |X_k - g|^2 = \sum_k \omega_k |X_k|^2 - |g|^2 = 1 - |g|^2$, where $g = \sum_k \omega_k X_k$; applied to the neighbors and the previous rotation, this is the last term. □

Each term has a plain meaning. The first is the membrane energy of the cell at its rotation: its ARAP distortion, which is zero for a cell that moves rigidly and small for one that deforms little. The second is the distance between the cell's rotation and the mean rotation of its neighborhood. The third is the spread of the neighborhood's rotations around their mean, its variance. In a smooth, nearly rigid deformation all three are small compared

with αA, so the true multiplier is close to the frozen one, and the linearized cell is close to the exact one. The proposition also says when they differ: where cells deform strongly, or where neighboring rotations disagree.

Which solution is the minimizer? The system (5.12) can have several solutions, with different multipliers, as the eigenvector equation does when $h_i = 0$. The following classical result [3,9] picks the right one. Let $m_1 \le m_2 \le m_3 \le m_4$ be the eigenvalues of M_i and $v_1, \dots, v_4$ its unit eigenvectors.

Proposition 5.7. *A solution (R_i^*, λ^*) of (5.12) with $\lambda^* \le m_1$ is a global minimizer of (5.11) on the unit sphere.*

Proof. For any unit quaternion S, write $G = M_i - \lambda^* I$, which is positive semi-definite since $\lambda^* \le m_1$. Using $h_i = GR_i^*$ and $|S| = |R_i^*| = 1$,

$$\begin{aligned} E_i(S) - E_i(R_i^*) &= \big(S^T G S + \lambda^* - 2R_i^{*T} G S\big) - \big(R_i^{*T} G R_i^* + \lambda^* - 2R_i^{*T} G R_i^*\big) \\ &= (S - R_i^*)^T G\,(S - R_i^*) \ge 0. \end{aligned}$$

Here we used $S^T M_i S = S^T G S + \lambda^* |S|^2$ and $h_i^T S = R_i^{*T} G S$. □

So we look for the multiplier below the smallest eigenvalue of M_i. Note that m_1 has a meaning of its own: its eigenvector v_1 is the rotation that fits the cell's edges best, the ARAP rotation of the cell alone, and m_1 is its residual.

The secular equation. For $\lambda < m_1$ the matrix $M_i - \lambda I$ is positive definite, and (5.12) has the unique solution $R(\lambda) = (M_i - \lambda I)^{-1} h_i$. Only its length depends on the choice of λ: writing $c_k = v_k \cdot h_i$ for the components of the pull in the eigenbasis,

$$|R(\lambda)|^2 = \sum_{k=1}^{4} \frac{c_k^2}{(m_k - \lambda)^2}.$$

As λ decreases from m_1 to $-\infty$, every term decreases, from $+\infty$ (if $c_1 \neq 0$) to 0, so exactly one $\lambda^* < m_1$ gives $|R(\lambda^*)| = 1$: the multiplier of the minimizer. Finding it is a one-dimensional root-finding problem, but $|R(\lambda)|$ has a pole at m_1, which makes Newton's method on $|R(\lambda)| - 1$ overshoot. The standard remedy [9] is to apply Newton's method to the reciprocal, $\psi(\lambda) = 1/|R(\lambda)|$, which is almost linear.

Proposition 5.8. *On $(-\infty, m_1)$, ψ is decreasing and concave. Newton's method for $\psi(\lambda) = 1$, started at any $\lambda^{(0)} \in [\lambda^*, m_1)$, produces a decreasing sequence that converges to λ^*.*

Proof. Let $\phi(\lambda) = |R(\lambda)|^2 = \sum_k c_k^2 t_k^{-2}$ with $t_k = m_k - \lambda > 0$, so that $\psi = \phi^{-1/2}$. Since $dt_k/d\lambda = -1$, $\phi' = 2\sum_k c_k^2 t_k^{-3} > 0$ and $\phi'' = 6\sum_k c_k^2 t_k^{-4}$; hence $\psi' = -\frac{1}{2}\phi^{-3/2}\phi' < 0$, and

$$\begin{aligned} \psi'' &= \tfrac{1}{4}\,\phi^{-5/2}\big(3\phi'^2 - 2\phi\,\phi''\big) \\ &= 3\,\phi^{-5/2}\Big(\big(\textstyle\sum_k c_k^2 t_k^{-3}\big)^2 - \big(\sum_k c_k^2 t_k^{-2}\big)\big(\sum_k c_k^2 t_k^{-4}\big)\Big) \le 0 \end{aligned}$$

by the Cauchy–Schwarz inequality applied to the vectors $(|c_k| t_k^{-1})_k$ and $(|c_k| t_k^{-2})_k$. Now let $\lambda^{(t)} \in (\lambda^*, m_1)$. There $\psi < 1$, so the Newton step $-(\psi - 1)/\psi'$ is negative and $\lambda^{(t+1)} < \lambda^{(t)}$. A concave function lies below its tangents, so at $\lambda^{(t+1)}$, where the tangent equals 1, $\psi(\lambda^{(t+1)}) \le 1$, that is $\lambda^{(t+1)} \ge \lambda^*$. The sequence is decreasing and bounded below by λ^*, so it converges, and its limit solves $\psi = 1$. □

The proposition is the mirror image of the usual argument for Newton's method on a convex function, and it needs a start between the root and the pole. One is at hand. Since $|R(\lambda)| \ge |c_1|/(m_1 - \lambda)$, the point $\lambda^{(0)} = m_1 - |c_1| = m_1 - |v_1 \cdot h_i|$ has $|R| \ge 1$, so it lies in $[\lambda^*, m_1)$. Each Newton step needs $|R|$ and ψ'; with a Cholesky factorization $M_i - \lambda I = LL^T$, which exists because the matrix is positive definite,

$$R = L^{-T} L^{-1} h_i, \qquad \psi'(\lambda) = -\frac{R^T (M_i - \lambda I)^{-1} R}{|R|^3} = -\frac{|L^{-1} R|^2}{|R|^3},$$

$$\lambda \leftarrow \lambda - \frac{\psi(\lambda) - 1}{\psi'(\lambda)} = \lambda + \frac{|R|^2 \, (1 - |R|)}{|L^{-1} R|^2}.$$

A 4×4 factorization and three triangular solves per step: the whole cell costs a few hundred floating-point operations, and Newton's method converges in two to four steps in our experiments (Section 9.7).

The smallest eigenpair. The start needs m_1 and v_1. They come from the characteristic polynomial $P(\lambda) = \det(M_i - \lambda I) = \prod_k (m_k - \lambda)$, whose coefficients follow from the traces of M_i, M_i^2 and M_i^3 and from $\det M_i$ by Newton's identities. On $(-\infty, m_1]$ every factor is non-negative, so P is positive, decreasing and convex there, and Newton's method started from any point below m_1 increases monotonically to m_1, by the same tangent argument. Since M_i is positive semi-definite, $\lambda = 0$ is such a point. The eigenvector follows from the adjugate: $(M_i - m_1 I) \operatorname{adj}(M_i - m_1 I) = P(m_1)\, I = 0$, so every column of the adjugate is a multiple of v_1, and the column k has length proportional to $|v_{1,k}|$. Taking the column with the largest diagonal entry, the largest coordinate of v_1, avoids the columns that vanish, and gives v_1 to full accuracy.

The hard case. When the pull is orthogonal to the cell's own rotation, $c_1 = v_1 \cdot h_i = 0$, the pole at m_1 disappears and $|R(\lambda)|$ stays bounded as $\lambda \to m_1$. If its limit $|y|$, with $y = \sum_{k \ge 2} c_k v_k / (m_k - m_1)$, is less than one, the multiplier stops at $\lambda^* = m_1$ and the minimizer is $R_i^* = y \pm \sqrt{1 - |y|^2}\, v_1$: the part along v_1 is no longer determined by the pull, only completed to unit length. This *hard case* of the trust-region literature [9] looks like a curiosity, but Section 5.9 shows that it is exactly what happens when a handle is turned by 180°.

Two limits. The exact cell interpolates between two familiar answers. Without bending, $\alpha A \to 0$, the pull vanishes, $\lambda^* \to m_1$, and $R_i^* \to \pm v_1$: the ARAP rotation of the cell, with the sign chosen by the side of the pull, which also resolves the ambiguity between R and $-R$. With very stiff bending, $\alpha A \to \infty$, the membrane term becomes negligible, $\lambda^* \approx -|h_i|$, and $R_i^* \to g_i/|g_i|$: the

normalized mean of the neighbors. The linearized cell has the same two limits whenever the neighbors agree, $|g_i| = 1$, which is another way to see why it works.

5.9. Constrained cells and large rotations

Section 5.5 gave the cells inside the handles the rotation of the cell equation without feedback, (5.8): the fit of their edges, which the handle moves rigidly, smoothed towards the previous rotations of their neighbors. That rule works well as long as the handles turn by moderate angles, and it fails, in a precise sense, when a handle is turned by a large angle at once: the cells of the handle do not follow the handle. This section explains why, and derives the rule we use instead. The explanation needs two notions, the cell's own rotation and the pull of its neighbors, and one fact about quaternions.

Notation. Throughout this section $i \in \mathcal{C}$ is a constrained cell, and $M_i = \sum_{k=1}^{4} m_k\, v_k v_k^T$ is the eigendecomposition of its membrane matrix, with $0 \le m_1 \le m_2 \le m_3 \le m_4$ and orthonormal v_k (we omit the index i on m_k and v_k).

Definition 5.9. *The* own fit *of cell i is the unit quaternion $v_1 = v_1(M_i)$, defined up to sign: the rotation that fits the cell's edges best, the minimizer of the membrane term $R^T M_i R$ on the unit sphere. The* pull *of its neighbors is the right-hand side of* (5.8)*,*

$$h_i = \alpha A\, \bar{g}_i, \qquad \bar{g}_i = \frac{1}{d_i} \sum_{j \in N(i)} R_j^{prev},$$

and its components in the eigenbasis are $c_k = v_k \cdot h_i$. The pull angle *of the cell is*

$$\theta_i = 2 \arccos \frac{|v_1 \cdot h_i|}{|h_i|} \in [0^\circ, 180^\circ]. \tag{5.14}$$

The pull angle has a geometric meaning, which the following elementary fact makes precise.

Proposition 5.10. *Let q_1 and q_2 be unit quaternions, and let $\theta \in [0^\circ, 180^\circ]$ be the angle of the rotation that takes the rotation of q_1 to that of q_2. Then $|q_1 \cdot q_2| = \cos(\theta/2)$, where $q_1 \cdot q_2$ is the Euclidean inner product of their coefficients in $\mathbb{R}^4$.*

Proof. The relative rotation is $\tilde{q}_1 q_2$. Writing $q_k = w_k + B_k$ as in Section 7, the scalar part of $\tilde{q}_1 q_2 = (w_1 - B_1)(w_2 + B_2)$ is $w_1 w_2 + B_1 \cdot B_2 = q_1 \cdot q_2$, since the scalar part of $-B_1 B_2$ is $B_1 \cdot B_2$. A unit quaternion that rotates by θ has scalar part $\pm \cos(\theta/2)$, the sign depending on which of the two quaternions of the rotation is taken. □

So θ_i is the angle, as a rotation, between the cell's own fit and the mean rotation of its neighbors (normalized). Two rotations 180° apart have *orthogonal* quaternions; this is the situation of a handle turned by 180°, whose cells' own fit is the handle's rotation, while their neighbors, outside the handle, are still at rest.

What the cell equation does with a large pull angle. Expanding the solution of (5.8) in the eigenbasis of M_i shows exactly how much the smoothing takes the cell away from its own fit.

Proposition 5.11. *Let $R = (M_i + \alpha A\, I)^{-1} h_i$ be the unnormalized solution of the cell equation (5.8), and $\beta \in [0°, 90°]$ the angle between R and the line of v_1 in $\mathbb{R}^4$, so that the normalized R differs from the own fit by a rotation of 2β. Then*

$$\frac{m_1 + \alpha A}{m_4 + \alpha A} \tan\frac{\theta_i}{2} \;\le\; \tan\beta \;\le\; \frac{m_1 + \alpha A}{m_2 + \alpha A} \tan\frac{\theta_i}{2}. \tag{5.15}$$

Proof. In the eigenbasis, $R = \sum_k \frac{c_k}{m_k + \alpha A} v_k$. Its component along v_1 has length $|c_1|/(m_1 + \alpha A)$, and its component orthogonal to v_1 has squared length $\sum_{k\ge 2} c_k^2/(m_k + \alpha A)^2$, which lies between $\sum_{k\ge 2} c_k^2/(m_4 + \alpha A)^2$ and $\sum_{k\ge 2} c_k^2/(m_2 + \alpha A)^2$. By the definition (5.14), $|c_1| = |h_i| \cos(\theta_i/2)$, and since the v_k are orthonormal, $\sum_{k\ge 2} c_k^2 = |h_i|^2 - c_1^2 = |h_i|^2 \sin^2(\theta_i/2)$. Dividing the orthogonal length by the parallel one gives $\tan\beta$ between the two bounds. □

What matters for the iteration is not the lag itself but how it changes from one local step to the next. At the start of a local step the cell sits, roughly, at the mean rotation of its neighbors, whose quaternion line makes the angle $\theta_i/2$ with that of the own fit (Proposition 5.10); the cell equation takes it to the angle β. The difference

$$\delta_i = \frac{\theta_i}{2} - \beta \tag{5.16}$$

is the *progress* of the cell towards its own fit during one local step, measured as an angle between quaternion lines (the rotation angle is twice as large). Proposition 5.11 bounds it between two functions of the same shape.

Proposition 5.12. *Let $r_2 = (m_1 + \alpha A)/(m_2 + \alpha A)$ and $r_4 = (m_1 + \alpha A)/(m_4 + \alpha A)$, both in $(0, 1]$, and for $r \in (0, 1]$ let $f_r(x) = x - \arctan(r \tan x)$ on $[0°, 90°]$. Then*

$$f_{r_2}(\theta_i/2) \;\le\; \delta_i \;\le\; f_{r_4}(\theta_i/2).$$

Each f_r vanishes at $x = 0°$ and at $x = 90°$, is positive in between when $r < 1$, and increases up to $\tan x = 1/\sqrt{r}$, then decreases. In particular the progress is zero at $\theta_i = 180°$, whatever the stiffness.

Proof. The bounds follow from (5.15) because arctan is increasing. Clearly $f_r(0) = 0$, and $r \tan x \to \infty$ as $x \to 90°$, so $\arctan(r \tan x) \to 90°$ and $f_r \to 0$. For $r < 1$, $r \tan x < \tan x$ gives $f_r > 0$ in between. Differentiating, $f_r'(x) = 1 - r(1 + \tan^2 x)/(1 + r^2 \tan^2 x)$, which vanishes when $1 + r^2 t^2 = r(1 + t^2)$ with $t = \tan x$, that is when $t^2(r - r^2) = 1 - r$, or $t^2 = 1/r$; f_r' is positive before that point and negative after it. □

On our meshes the bending stiffness dominates the cells' own stiffness, $m_k \ll \alpha A$: the ratios r_2 and r_4 of the handle cells of the bar lie between 0.985 and 0.999, and on the Armadillo's control mesh, whose cells are larger, r_4 goes down to 0.38 (Section 9.7). The cell equation then returns almost the

mean of the neighbors, and moves the cell only a small step towards its own fit in each local step. The handle reaches its cells through those small steps, and through the global step, iteration after iteration. Proposition 5.12 says how the size of the step depends on the angle: it grows with θ_i up to about $90°$ (exactly, up to $\tan(\theta_i/2) = 1/\sqrt{r}$, which is $90°$ for $r = 1$ and a little more for $r < 1$), and beyond that it shrinks, until at $\theta_i = 180°$ it vanishes. A handle turned by $180°$ at once is therefore the worst case: its cells do not move towards the handle at all, and they stay at the rotation of their neighbors. The exact cell on the sphere (Section 5.8) does not help here: at $\theta_i = 180°$ the pull meets the condition $c_1 = 0$ of the hard case, and the exact cell agrees with the linearized one to 10^{-3} degrees in our experiments (Table 4).

The consequence for the deformation is visible in Figure 9(a): when a handle is turned by $180°$ at once, its cells stay close to their neighbors' rotation, and the bar is pinched just below the handle. The rotation field is in a balanced but unstable configuration, which it leaves only when round-off and the slow change of the positions break the symmetry; on the bar this takes between 1,000 and 2,000 local/global iterations (Section 9.7).

The adaptive rule. The diagnosis suggests the remedy. The rotation of a constrained cell is not an unknown of the local step, and nothing obliges us to smooth it: the smoothing exists to make the handle blend into its surroundings, and Propositions 5.11 and 5.12 show that beyond $\theta_i = 90°$ it does the opposite of blending: the further the pull is from the own fit, the more slowly the cell is brought to its handle, and at $180°$ it is not brought at all. We therefore keep the cell equation where it behaves well and take the own fit where it does not:

$$R_i^{const} = \begin{cases} \dfrac{(M_i + \alpha A\, I)^{-1} h_i}{|(M_i + \alpha A\, I)^{-1} h_i|} & \text{if } \theta_i \le \theta_{\max}, \\ \sigma_i\, v_1(M_i) & \text{if } \theta_i > \theta_{\max}, \end{cases} \qquad \theta_{\max} = 90°. \tag{5.17}$$

Three choices in (5.17) need a justification.

The threshold. By Proposition 5.12, the progress of the cell equation towards the own fit is largest near $\theta_i = 90°$ and decreases beyond it, to zero at $180°$; the further the pull is beyond $90°$, the slower the cell equation brings the cell to its handle. $\theta_{\max} = 90°$ is therefore the angle beyond which the cell equation stops being the fastest way for the cell to reach its own fit, and taking the own fit directly can only help. In $\mathbb{R}^4$ it is the angle at which the pull makes $45°$ with the line of the own fit: it is as close to the own fit as to the space orthogonal to it. The experiments of Section 9.7 show that the choice is not critical.

The sign. A quaternion and its opposite represent the same rotation, but the bending term $\|R_i - R_j\|^2$ of (5.2) compares them as vectors of $\mathbb{R}^4$, so neighboring cells must use quaternions on the same side. Choosing $\sigma_i = \pm 1$ from the cell's previous rotation, as elsewhere, fails precisely here: when $\theta_i > 90°$ the previous rotation is nearly orthogonal to v_1, and the sign of $v_1 \cdot R_i^{prev}$ is decided by round-off, differently in neighboring cells. Instead we

take σ_i so that $\sigma_i\, v_1 \cdot q_H \geq 0$, where q_H is the quaternion of the handle that contains the vertex. The handle is turned by the user in small increments, so q_H moves continuously and stays on one branch, and all the cells of a handle get the same side.

The fixed point. The rule changes the path of the iterations, not their destination.

Proposition 5.13. *Let* (P, R) *be a state of the local/global iteration at which every constrained cell has* $\theta_i \leq \theta_{\max}$*. Then the cell equation and the adaptive rule (*5.17*) give the same constrained rotations at* (P, R)*, and hence the same local and global steps. In particular, such a state is a fixed point of the iteration with the adaptive rule if and only if it is a fixed point with the cell equation.*

Proof. For $\theta_i \leq \theta_{\max}$ the first case of (5.17) is (5.8) itself. The constrained rotations are the only place where the two iterations differ, so they produce the same rotation system (5.9), the same rotations, and the same positions. □

At the fixed points we reach, every handle cell has its own fit within 27° of its neighbors' mean, far below $\theta_{\max}$ (Section 9.7), so the proposition applies: the adaptive rule converges to the same shapes as the cell equation, only sooner. When the handles turn gradually, as in interactive editing, the neighbors follow the handle from one frame to the next, the pull angle stays small, and the rule never departs from the cell equation.

Cost. The rule needs the own fit v_1 of every constrained cell, which is the smallest eigenpair of a 4×4 matrix, computed with the Newton iteration and the adjugate column of Section 5.8, a few dozen operations per cell; the pull h_i is already on the right-hand side of (5.8). On the bar, with 1,898 constrained cells, the rule adds 0.6 ms to a PCG local step of 14.7 ms.

6. Multiresolution Optimization

To achieve interactive rates, we optimize the SR-ARAP energy on a low-resolution version of the input mesh, the *control mesh*, and then transfer the solution to the full-resolution mesh. The control mesh is obtained by simplifying the input mesh with *half-edge collapses* that minimize the quadric error metric, so that it is a triangulation of a subset of the original vertices. Each control vertex k is associated with a full-resolution vertex $h(k)$: the vertex it was kept from, or equivalently its nearest full-resolution vertex.

Once the optimal deformed shape of the control mesh is found, the optimized rotations are transferred to the full-resolution mesh by harmonic interpolation of quaternions (see Section 6.1). The full-resolution positions are then obtained from Equation (4.1) on the full-resolution mesh, with the interpolated rotations and with the optimized control vertices as positional constraints, $p_{h(k)} = p_k^{control}$:

$$\sum_{j \in N(i)} c_{ij}\,(p_j - p_i) = \sum_{j \in N(i)} \frac{c_{ij}}{2}\left(R_i\, q_{ij}\, \tilde{R}_i + R_j\, q_{ij}\, \tilde{R}_j\right).$$

Algorithm 1 summarizes one interactive frame.

Algorithm 1 One interactive frame. The Cholesky factorizations of Sections 4 and 6.1, and that of the preconditioner C of Section 5.7, are computed once per constraint set.

1: Set the positional constraints p_i^{const} of the control mesh from the handle transformations.
2: **for** $k = 1$ to K **do**
3: Compute M_i for every control vertex from the current positions (Section 7).
4: Compute R_i^{const}, $i \in \mathcal{C}$, by the adaptive rule (5.17); solve (5.9) for R_i, $i \in \mathcal{F}$, by PCG from the previous rotations; normalize all R_i.
5: Solve the global step (4.2) on the control mesh.
6: **end for**
7: Interpolate the control rotations onto the full-resolution mesh (6.4) and normalize them.
8: Solve the global step (4.2) on the full-resolution mesh, constrained to the control positions.

6.1. Harmonic Interpolation of Quaternions

The seminal work of Pinkall and Polthier [11] shows how to interpolate given data over a discrete domain using harmonic maps. Harmonic maps are critical points of the Dirichlet (stretching) energy $\int_\Omega |\nabla f|^2 \, dA$, which generates minimal surfaces. The *discrete harmonic energy* of a map f defined on the mesh vertices $\{q_i\}$ has the form

$$E(f) = \sum_{\{i,j\} \in \mathcal{E}} c_{ij} \, \|f(q_j) - f(q_i)\|^2, \tag{6.1}$$

where c_{ij} are the symmetric cotangent weights [11] of the edges. The discrete Laplacian operator can be identified in that energy as

$$\Delta f(q_i) = \sum_{j \in N(i)} c_{ij} \big(f(q_j) - f(q_i)\big). \tag{6.2}$$

Harmonic energy is known to minimize angular distortion, which means that harmonic functions blend boundary conditions smoothly over the domain. Harmonic functions are intrinsic to surfaces and independent of the discretization used to produce the meshes. In a spirit similar to [14], we interpolate quaternions over a mesh with harmonic functions. The harmonic quaternion interpolation over a surface mesh is the solution of the discrete harmonic equation

$$\sum_{j \in N(i)} c_{ij} \, (R_j - R_i) = 0 \tag{6.3}$$

subject to Dirichlet boundary conditions $R_{h(k)} = R_k^{control}$, where $\{R_k^{control}\}$ are the optimized quaternions of the control mesh. This interpolation produces a smooth field of quaternions by "averaging" the boundary conditions

gradually over the surface, in effect a linear interpolation of the boundary conditions across the surface. In matrix form,

$$\mathrm{L}\,\mathrm{R} = 0 \qquad \text{s.t. } R_{h(k)} = R_k^{control},$$

where L is the full-resolution Laplacian of the global step and R is the $m \times 4$ matrix of quaternion coefficients. Substituting the boundary conditions as in Section 4, with $\mathcal{C} = \{h(k)\}$, gives

$$(-\mathrm{L}_{\mathcal{F}\mathcal{F}})\,\mathrm{R}_{\mathcal{F}} = \mathrm{L}_{\mathcal{F}\mathcal{C}}\,\mathrm{R}_{\mathcal{C}}^{control}, \tag{6.4}$$

which has the same matrix as the full-resolution global step, whose constrained vertices are the same set $\{h(k)\}$. One Cholesky factorization therefore serves both the rotation interpolation, with four right-hand sides, and the full-resolution positions, with three. The interpolated quaternions are not of unit length and must be normalized afterwards. Despite its simplicity, this approach works surprisingly well, is extremely efficient, and provides a natural propagation at no extra cost.

7. The form of M_i

As noted in Section 5, for a unit quaternion R_i the membrane term equals

$$\sum_j c_{ij}\,\|R_i\,q_{ij}\,\tilde{R}_i - p_{ij}\|^2 = \sum_j c_{ij}\,\|R_i\,q_{ij} - p_{ij}\,R_i\|^2.$$

We now compute the matrix M_{ij} that represents $R_i\,q_{ij} - p_{ij}\,R_i$ and the matrix M_i of the cell.

Expanding the products. Write $R_i = w_i + B_i$, with scalar part w_i and pure quaternion part B_i. For pure quaternions u and v the quaternion product is $u\,v = -u \cdot v + u \times v$, a scalar part and a vector part. Since the scalar w_i commutes with everything,

$$R_i\,q_{ij} = -B_i \cdot q_{ij} + w_i\,q_{ij} + B_i \times q_{ij}, \qquad p_{ij}\,R_i = -p_{ij} \cdot B_i + w_i\,p_{ij} + p_{ij} \times B_i.$$

Subtracting, and using $B \times q + B \times p = -(q+p) \times B$,

$$R_i\,q_{ij} - p_{ij}\,R_i = -\,a_{ij} \cdot B_i + w_i\,a_{ij} - b_{ij} \times B_i, \qquad a_{ij} = q_{ij} - p_{ij}, \quad b_{ij} = q_{ij} + p_{ij}, \tag{7.1}$$

where the first term is the scalar part and the last two are the vector part.

The matrix M_{ij}. Representing R_i as the 4×1 column vector made of the scalar w_i and the 3×1 column vector B_i, the same expression in matrix form is

$$M_{ij}R_i = \begin{bmatrix} 0 & -a_{ij}^T \\ a_{ij} & -[b_{ij}]_\times \end{bmatrix} \begin{bmatrix} w_i \\ B_i \end{bmatrix} = \begin{bmatrix} -a_{ij}^T B_i \\ w_i\,a_{ij} - b_{ij} \times B_i \end{bmatrix}, \tag{7.2}$$

where a_{ij} and b_{ij} are 3×1 column vectors and $[b]_\times$ is the skew-symmetric cross-product matrix, $[b]_\times\,x = b \times x$. Since $[b_{ij}]_\times$ is skew-symmetric and the two off-diagonal blocks are $-a_{ij}^T$ and a_{ij}, M_{ij} is a skew-symmetric 4×4 real matrix, $M_{ij}^T = -M_{ij}$.

The product $M_{ij}^T M_{ij}$. Multiplying the blocks of M_{ij}^T and M_{ij}, and using $-a^T[b]_\times = ([b]_\times a)^T = (b \times a)^T$,

$$M_{ij}^T M_{ij} = \begin{bmatrix} \|a_{ij}\|^2 & (b_{ij} \times a_{ij})^T \\ b_{ij} \times a_{ij} & a_{ij}a_{ij}^T - [b_{ij}]_\times^2 \end{bmatrix}. \tag{7.3}$$

It is symmetric and positive semi-definite, as every matrix of the form $X^T X$, and since $c_{ij} \geq 0$ so is the sum $M_i = \sum_j c_{ij}\, M_{ij}^T M_{ij}$ of (5.1): all its eigenvalues are real and non-negative. Its trace, used in Section 5.7, is $\operatorname{tr} M_{ij}^T M_{ij} = 2\|a_{ij}\|^2 + 2\|b_{ij}\|^2 = 4(\|p_{ij}\|^2 + \|q_{ij}\|^2)$, since $\operatorname{tr}(-[b]_\times^2) = 2\|b\|^2$.

7.1. Efficient Computation of M_i

Since $[b]_\times^2 = b\,b^T - \|b\|^2 I$, the product $M_{ij}^T M_{ij}$ has the simple form

$$M_{ij}^T M_{ij} = \begin{bmatrix} \|a_{ij}\|^2 & (b_{ij} \times a_{ij})^T \\ b_{ij} \times a_{ij} & a_{ij}a_{ij}^T - b_{ij}b_{ij}^T + \|b_{ij}\|^2 I_{3\times 3} \end{bmatrix}.$$

Writing it in terms of q_{ij} and p_{ij}, with $\|a_{ij}\|^2 = \|q_{ij}\|^2 + \|p_{ij}\|^2 - 2\,q_{ij}^T p_{ij}$, $\|b_{ij}\|^2 = \|q_{ij}\|^2 + \|p_{ij}\|^2 + 2\,q_{ij}^T p_{ij}$, $b_{ij} \times a_{ij} = -2\,q_{ij} \times p_{ij}$ and $a_{ij}a_{ij}^T - b_{ij}b_{ij}^T = -2\,(q_{ij}p_{ij}^T + p_{ij}q_{ij}^T)$, gives

$$\begin{aligned} M_{ij}^T M_{ij} = & -2 \begin{bmatrix} q_{ij}^T p_{ij} & (q_{ij} \times p_{ij})^T \\ q_{ij} \times p_{ij} & q_{ij}p_{ij}^T + p_{ij}q_{ij}^T - q_{ij}^T p_{ij}\, I_{3\times 3} \end{bmatrix} \\ & + (\|p_{ij}\|^2 + \|q_{ij}\|^2)\, I_{4\times 4}. \end{aligned} \tag{7.4}$$

All terms of (7.4) can be derived from the dyadic tensor $q_{ij}p_{ij}^T$ and the scalar $\|p_{ij}\|^2 + \|q_{ij}\|^2$. Summing over the cell, M_i is obtained from the single 3×3 matrix $T_i = \sum_j c_{ij}\, q_{ij}p_{ij}^T$ and the scalar $\sigma_i = \sum_j c_{ij}(\|p_{ij}\|^2 + \|q_{ij}\|^2)$: the trace of T_i gives $\sum_j c_{ij}\, q_{ij}^T p_{ij}$, its antisymmetric part gives $\sum_j c_{ij}\, q_{ij} \times p_{ij}$, and $T_i + T_i^T$ gives $\sum_j c_{ij}(q_{ij}p_{ij}^T + p_{ij}q_{ij}^T)$. Computing M_i therefore costs one 3×3 outer-product accumulation per edge.

8. Implementation

The author's source code is publicly available on GitHub [8]. The implementation divides Equations (5.4) and (5.8) by two, working with $\frac{1}{2}M_i$ and $\frac{\alpha}{2}$. It uses $\alpha = 0.28$ (that is, $\frac{\alpha}{2} = 0.14$) and $e = 0.2$, and adds $\eta = 2 \cdot 10^{-6}$ to the diagonal of every block $M_i + \alpha A\, I$ for numerical safety, which adds the Tikhonov term $\eta \sum_i (d_i + e)\, \|R_i\|^2$ to (5.6). The constrained rotations follow the adaptive rule (5.17) with $\theta_{\max} = 90°$. Each interactive frame performs $K = 3$ local/global iterations on the control mesh. The local step is solved by PCG (Section 5.7), warm-started from the previous rotations and stopped at a relative residual of 10^{-12}; its preconditioner C is factored when the constraints are set and reused afterwards. Convergence is monitored with the energy (3.1), with $w_{ij} = 1/d_i$.

9. Results

Setup. All measurements were taken on an Apple M1 Pro laptop (10 cores, 32 GB), with a port of the reference implementation [8] to Rust that uses the sparse Cholesky factorization of the `faer` library; every factorization and solve runs on a single thread, except that the right-hand sides of a Poisson solve are solved in parallel. The local step is solved by PCG with the factor-once preconditioner (Section 5.7); we call its solution, and that of a direct sparse Cholesky factorization of $\mathrm{K}_{\mathcal{FF}}$, the closed form, since the two agree to about 10^{-12} rad. Unless stated otherwise, $\alpha = 0.28$, $e = 0.2$ and $K = 3$ local/global iterations per frame (Section 8). The models are the Armadillo (172,974 vertices) and a cactus (5,261 vertices), each with a 502-vertex control mesh, and a tube, the classic bar benchmark of ARAP papers, generated at resolutions from 530 to 124,010 vertices; its ends are capped by concentric rings so that no vertex has more than 12 neighbors. The relaxation baseline solves exactly the same per-cell equations (5.4): each sweep visits every free cell once and solves its 4×4 system with the current neighbor rotations (block Gauss–Seidel), and the quaternions are normalized after the last sweep. By Proposition 5.1, relaxation and the closed form share the same solution; with $e = 0.2$ we verified that relaxation reaches it to machine precision (10^{-15} rad) on every mesh.

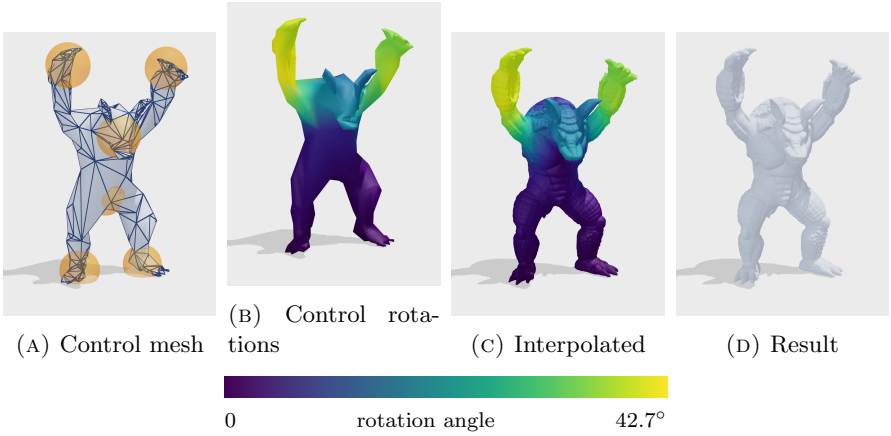
9.1. Interactive multiresolution deformation

Figure 1 shows poses of the Armadillo produced with the multiresolution scheme, and Figure 2 its stages: the rotations found on the control mesh are carried smoothly to the full surface by the harmonic interpolation, with no visible seams at the coarse cells. Table 1 breaks one interactive frame into its stages. With a 502-vertex control mesh, one local step costs 0.9–1.1 ms with PCG (15 to 21 iterations) and 0.5–1.3 ms with a direct factorization; at this size the two are comparable. On the Armadillo the three local steps of a frame take 2.6 ms of 30.4 ms, and the frame is dominated by the two full-resolution solves of Section 6.1, which share one Cholesky factorization; on the cactus the local steps take most of a 4.8 ms frame. In this setting the exact solution of the local step costs little.

The symmetric reduction of Sections 4 and 5.4 matters in practice. In the reference C++ implementation (Eigen), replacing the sparse LU of the row-replaced systems by Cholesky factorizations of the reduced systems lowers the frame time from 18.2 to 6.6 ms on the cactus and from 266 to 182 ms on the Armadillo, with results identical to 10^{-13}; solving the rotation system by PCG instead lowers it further to 6.1 ms on the cactus, with rotations within $2.4 \cdot 10^{-12}$ rad of the LU solution.

9.2. Comparison with SR-ARAP

We compare against the original SR-ARAP local step of Levi and Gotsman [6], run with the same global step, constraints and multiresolution transfer. It



(A) Control mesh (B) Control rotations (C) Interpolated (D) Result

FIGURE 2. The multiresolution pipeline on the Armadillo, for the pose of Figure 1(c). (a) The deformed 502-vertex control mesh and its handles. (b) The rotation angle of every control cell after the closed-form local step. (c) The rotation field interpolated harmonically onto the 172,974 vertices of the full mesh. (d) The full-resolution surface reconstructed from the interpolated rotations, with the control vertices as constraints.

TABLE 1. Cost of one interactive frame, in milliseconds: the local step on the control mesh with PCG, and for comparison with a direct Cholesky factorization and with 3 relaxation sweeps; the global step (local and global steps are each performed $K = 3$ times per frame); the harmonic interpolation of the rotations; and the full-resolution positions.

Model	Control	Full	PCG	Direct	Relax. 3	Global	Transfer	Detail
Cactus	502	5,261	1.10	1.30	0.08	0.07	0.48	0.26
Armadillo	502	172,974	0.87	0.55	0.12	0.08	15.27	12.80

Frame: 4.8 ms (cactus) and 30.4 ms (Armadillo).

keeps exact rotation matrices: each cell's rotation maximizes $\operatorname{tr}(R_i^T S_i)$ with

$$S_i = \sum_{j \in N(i)} c_{ij}\, p_{ij} q_{ij}^T + \alpha' A \sum_{j \in N(i)} (w_{ij} + w_{ji})\, R_j,$$

found by a singular value decomposition (orthogonal Procrustes) with the neighboring rotations fixed, and cells are visited in Gauss–Seidel sweeps. Each update therefore minimizes the total SR-ARAP energy, with bending term $\alpha' A \sum_i \sum_j w_{ij} \|R_i - R_j\|_F^2$, exactly over one cell. We use $w_{ij} = 1/d_i$ and $\alpha' = \alpha/8$, which gives both bending terms the same stiffness for small rotation

differences ($\|R_i - R_j\|_F^2 \approx 2\theta^2$ and $\|q_i - q_j\|^2 \approx \theta^2/4$ for rotations θ apart). The two methods do not minimize the same energy, so we compare the resulting shapes with two measures that depend only on the deformed positions: the ARAP distortion $\sum c_{ij}\|R_i q_{ij} - p_{ij}\|^2 / \sum c_{ij}\|q_{ij}\|^2$ of the best rigid fit R_i of every cell, and the RMS angle between the fitted rotations of adjacent cells (roughness).

Table 2 and Figure 3 report both measures on the bar bent into a 90° arc and on the Armadillo, against the wall-clock time of the local/global iterations. The closed form converges to shapes with lower distortion and smoother rotations: 12% lower distortion and 1.18° against 1.37° roughness on the bar, and 5% lower distortion on the Armadillo. SR-ARAP with 3 and with 10 sweeps per iteration converges to the same shape, so the difference does not come from the sweep budget. The shapes differ by up to 2.6% of the model's half-diagonal on the bar and 1.6% on the Armadillo; Figure 11 shows where. Our method also gets there sooner. Measured by the wall-clock time to come within 1% of the distortion SR-ARAP converges to, PCG takes 3.5 s on the bar, against 7.4 s for SR-ARAP with 3 or with 10 sweeps per iteration (2.1 times faster), and 5.6 ms on the Armadillo, against 24 ms for SR-ARAP with 10 sweeps and 28–30 ms with 1 or 3 (4.3 to 5.3 times faster); it then continues to the lower distortion that SR-ARAP does not reach. Counted in iterations rather than time, the picture is more nuanced: on the Armadillo the closed form is ahead from the fifth iteration on, but on the bar, which undergoes much larger rotations, it leads for the first 50 iterations and again from the 230th on; in between SR-ARAP with 10 sweeps is ahead, and it settles within 1% of its final distortion after 160 iterations, against 540 for ours, each of its iterations costing three times as much. With a direct factorization of $\mathrm{K}_{\mathcal{F}\mathcal{F}}$ in every iteration instead of PCG, the bar would take 18.2 s: the speed-up over SR-ARAP relies on the iterative solver of Section 5.7. We attribute the slower start to the feedback term, which pulls every rotation towards its previous value: it provides temporal coherence and conditioning, at the price of a slower outer loop. The lower distortion does not come from the feedback: without it ($e = 0$) the closed form converges to an even lower distortion on the bar, $3.38 \cdot 10^{-3}$.

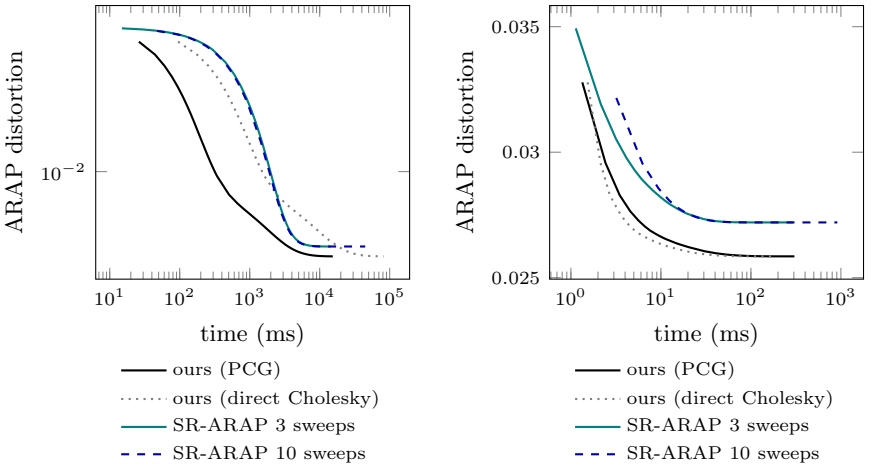
An SR-ARAP sweep costs one 3×3 SVD per cell, about ten times a sweep of the linearized relaxation (4.8 against 0.6 ms on the bar). A PCG local step costs about as much as three SR-ARAP sweeps (14.3 against 13.5 ms on the bar, 0.88 against 0.97 ms on the Armadillo's control mesh), but it is exact. Inexact relaxation of our linearized system with 3 sweeps per iteration is faster still (0.6 s on the bar), but its rotations are not exact.

9.3. Closed form and relaxation

Figure 4(a) shows how far relaxation is from the closed-form solution after each sweep, in the hardest case for relaxation: the first local step after the handle of a tube is bent by 60°, with all rotations still at rest. With the budget of two or three sweeps that interactive SR-ARAP implementations

TABLE 2. Our local step (PCG) against the original SR-ARAP local step after convergence (1,000 iterations on the bar, 300 on the Armadillo). Time is one local step; distortion and roughness are measured on the best rigid fit of every cell; gap is the largest distance to our shape, relative to the half-diagonal.

Model	Local step	Time (ms)	Distortion	Roughness	Gap
Bar, 90° arc	ours (PCG)	14.3	$3.42 \cdot 10^{-3}$	1.18°	–
	SR-ARAP, 3 sweeps	13.5	$3.87 \cdot 10^{-3}$	1.37°	2.6%
	SR-ARAP, 10 sweeps	43.8	$3.87 \cdot 10^{-3}$	1.37°	2.6%
Armadillo	ours (PCG)	0.88	$2.59 \cdot 10^{-2}$	7.56°	–
	SR-ARAP, 3 sweeps	0.97	$2.72 \cdot 10^{-2}$	7.68°	1.6%
	SR-ARAP, 10 sweeps	2.94	$2.72 \cdot 10^{-2}$	7.68°	1.6%

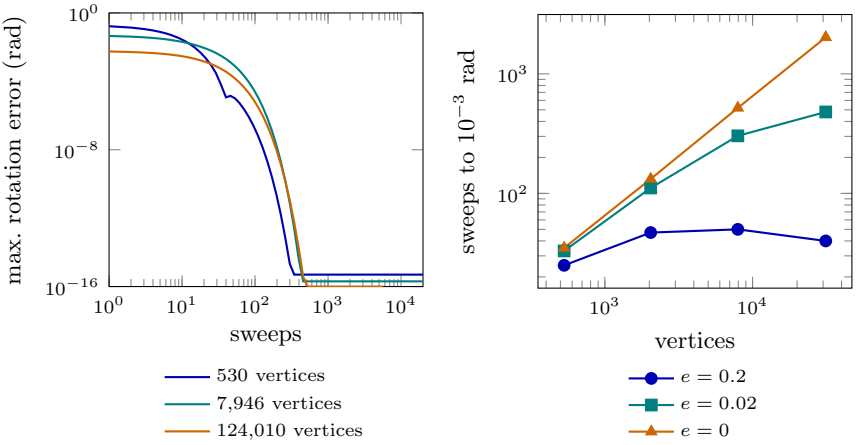


(A) Bar, 90° arc (B) Armadillo, 502-vertex control mesh

FIGURE 3. ARAP distortion of the shape against the wall-clock time of the local/global iterations, with our local step (solved by PCG, or by a direct factorization in every iteration) and with the original SR-ARAP local step.

use, the rotations are off by up to 0.11 rad (6.6°) on the coarsest tube and $5 \cdot 10^{-3}$ rad on the finest; the closed form has no such error.

The number of sweeps needed to reach a given accuracy, however, does not grow with the mesh: 25 to 50 sweeps reach 10^{-3} rad at every resolution. The reason is the feedback term. By Proposition 5.3, the eigenvalues of $K_{\mathcal{FF}}$ are bounded below by e, independently of the mesh, so the system is well conditioned and relaxation contracts at a mesh-independent rate. Figure 4(b) confirms this: with $e = 0.2$ the sweep count stays flat, while with weaker



(A) Relaxation error, cold start (B) Effect of the feedback weight e

FIGURE 4. Relaxation converges to the closed-form solution (a), but with a budget of a few sweeps it leaves rotation errors of several degrees. The number of sweeps it needs is independent of the mesh with the feedback term and grows linearly without it (b). Tube bent by 60°, first local step after the handle moves.

feedback it grows, and without feedback ($e = 0$, the plain smooth-rotation system) it grows linearly with the number of vertices, from 35 sweeps at 530 vertices to 2,022 at 31,274.

9.4. Solving the rotation system: direct and iterative

Table 3 and Figure 5 compare three ways of computing the same exact local step on the bent tube, in the cold start of Figure 4: a direct sparse Cholesky factorization of $K_{\mathcal{FF}}$ in every local step (with the symbolic analysis reused), conjugate gradients without a preconditioner, and PCG with the factor-once preconditioner C of Section 5.7. Both iterative solvers start from the previous rotations and are run until their rotations are within 10^{-12} rad of the direct solution.

The direct factorization grows superlinearly, roughly as $n^{1.5}$: from 1.8 ms at 530 vertices to 84 ms at 7,946 and 5.7 s at 124,010. Conjugate gradients needs 59 to 77 iterations at every resolution, as the first bound of Proposition 5.5 predicts, and its cost grows linearly, as $n^{0.94}$, reaching 0.31 s at 124,010 vertices. PCG needs even fewer iterations as the mesh is refined, 18 at 530 vertices and 5 at 124,010, as the second bound predicts; with each iteration costing one back-substitution, it solves the local step in 1.1 ms at 530 vertices and 0.11 s at 124,010, 1.7 to 49 times faster than the direct factorization. The factorization of C itself takes 0.4 to 110 ms and is done once

TABLE 3. Time of one exact local step on the tube (cold start), in milliseconds, with the number of iterations of the iterative solvers. Iterative solvers stop when their rotations are within 10^{-12} rad of the direct solution; their times include the assembly of the system. The factorization of the PCG preconditioner, done once per constraint set, is listed separately.

Vertices	Direct Cholesky	CG	PCG	Factor C (once)
530	1.8	1.8 (59)	1.1 (18)	0.4
2,042	12.0	4.4 (72)	2.0 (12)	1.1
7,946	84.1	19.0 (75)	7.0 (8)	7.5
31,274	657.0	80.0 (77)	27.3 (6)	20.1
124,010	5,677	308.6 (74)	114.9 (5)	109.9

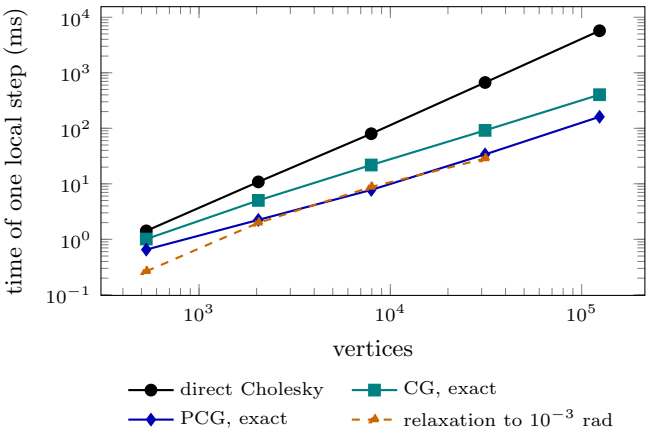
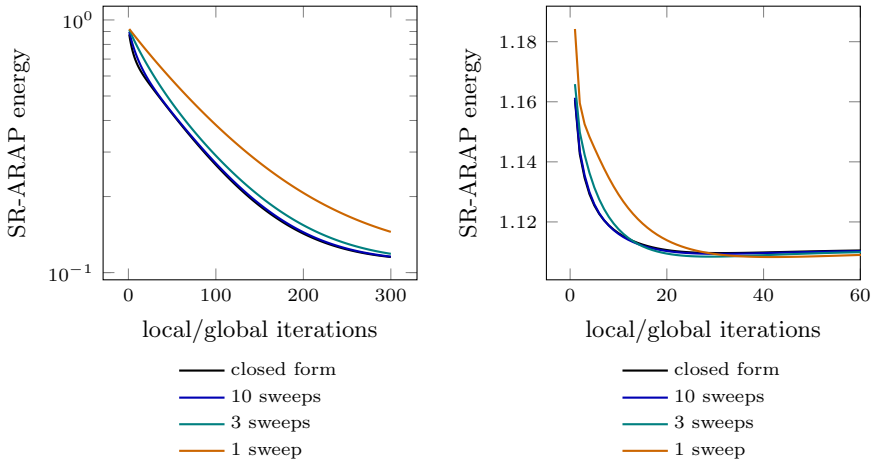


FIGURE 5. Time of one local step on the tube against its number of vertices. The direct factorization grows as $n^{1.5}$; conjugate gradients, which needs a mesh-independent number of iterations, grows linearly; PCG with the factor-once preconditioner needs fewer iterations as the mesh grows, is the fastest exact solver at every size, and on the larger tubes is as fast as inexact relaxation.

per constraint set. From about 8,000 vertices on, PCG reaches the exact solution in the time relaxation needs for an accuracy of 10^{-3} rad (Figure 5); on the smallest tubes relaxation to that accuracy remains a few times cheaper; preconditioning with the inverses of the 4×4 diagonal blocks (block Jacobi) brings no improvement over unpreconditioned conjugate gradients.

With feedback, relaxation also converges in a mesh-independent number of sweeps, but only to the accuracy it is given time for. Without feedback



(A) Tube, 7,946 vertices (B) Armadillo, 502-vertex control mesh

FIGURE 6. SR-ARAP energy per local/global iteration, with the closed-form local step and with relaxation limited to 1, 3 or 10 sweeps per iteration.

($e = 0$) its sweep count grows linearly with the mesh (Figure 4(b)), while the second bound of Proposition 5.5 no longer holds; the first bound degenerates as $e \to 0$ as well, and a direct factorization, which needs no bound on the conditioning, remains available.

9.5. Effect on the optimization

Figure 6 follows the SR-ARAP energy (3.1) through 300 local/global iterations on the bent tube (7,946 vertices) and on the Armadillo. On the tube, the closed form gives the lowest energy at every iteration: after 30 iterations the energy is 0.524 with the closed form, against 0.531, 0.594 and 0.693 with 10, 3 and 1 relaxation sweeps per iteration; after 300 iterations relaxation with one sweep is still 26% above the closed form. On the 502-vertex control mesh of the Armadillo, the closed form leads during the first ten iterations, after which all methods agree to within 0.1% (0.04% after 300 iterations). Since the closed form minimizes $\tilde{E}$ rather than E (Proposition 5.1), relaxation with a few sweeps can end slightly below it in E.

9.6. Feedback and large twists

To see what the feedback contributes for large rotations, we twist the top of the bar by 270°. A handle rotated by 270° places its vertices exactly where a rotation by −90° would, so its vertex positions cannot tell the two apart. Its quaternion can: $q_{270^\circ} = \cos 135^\circ + \sin 135^\circ\, n = -(\cos 45^\circ - \sin 45^\circ\, n) = -q_{-90^\circ}$ for the axis n, the same rotation with the opposite sign, and the bending term, which compares quaternions in $\mathbb{R}^4$, sees the difference. Set at once, the handle is 90° away, as a rotation, from its resting neighbors

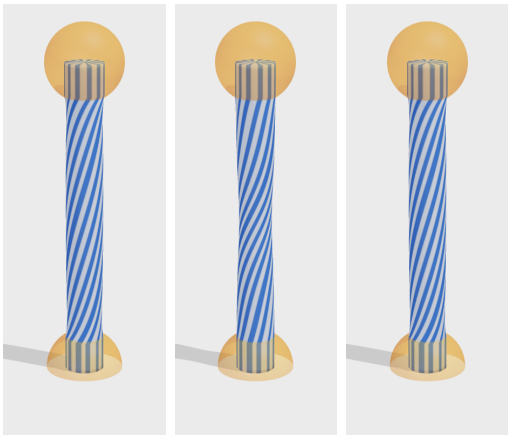
(the rotations by 270° and by −90° coincide), so its cells start right at the threshold of the adaptive rule: after the first global step their pull angles reach 95.6°. Those above 90° take their own fit with the sign of the handle's quaternion, $q_{270°}$, and the twist follows the 270° branch (Figure 7(c)). This outcome is not robust: with $\theta_{\max} = 100°$, or with the cell equation alone, no cell takes the handle's quaternion and the solver produces the −90° twist, which is an equally valid answer to a request that the positions alone cannot disambiguate. In interactive use the handle reaches 270° gradually. We turn it by 3° per step, with 20 local/global iterations per step, and then hold it at 270° for another 1,000 iterations, with and without feedback.

With feedback ($e = 0.2$) the twist follows the handle all the way to 270° and spreads evenly along the bar: its angle grows almost linearly with height, from 19° in the lowest tenth of the bar to 251° in the highest (Figure 7(a) and (d)). Without feedback ($e = 0$) the rotations also follow the 270° branch, since they are warm-started from the previous step, but the twist bunches in the middle of the bar: between 35% and 65% of the height it grows by 144°, against 85° with feedback and 81° for a uniform twist. The feedback thus does more than keep the branch: by anchoring every cell to its previous rotation, it lets the twist distribute along the bar as the handle turns, instead of concentrating where the rotations change fastest.

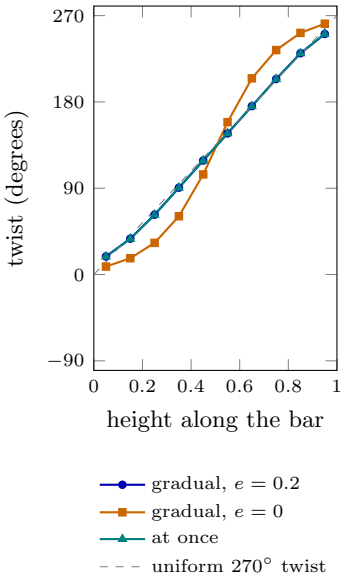
9.7. Exact cells and large rotations

How good is the frozen multiplier? Table 4 compares, cell by cell, the linearized cell equation with the exact cell on the unit sphere of Section 5.8, at the same state: five local/global iterations into the solves of the bar bent into a 90° arc, of the bar twisted by 180°, and of the Armadillo; and, for the twisted bar, also the cells inside the handle at the first local step, right after the handle has turned. The true multiplier is within 0.03% of the frozen value $-\alpha A$ in the median free cell and within 0.4% in the median handle cell, and within 3% in the worst cell, the Armadillo's most deformed one, as Proposition 5.6 predicts for nearly rigid cells. The rotations are closer still: the normalized linearized cell differs from the exact one by at most 0.15°, and in the median cell by less than 10^{-4} degrees. The same holds in the hard case at the handle, where the pull is orthogonal to the cell's own rotation. The linearization therefore does not change the cell problem in any significant way, which confirms with an exact reference what Section 5.1 assumed. The exact solve itself is cheap, two to three Newton steps for the multiplier after three to five for the smallest eigenvalue: three Gauss–Seidel sweeps of exact cells cost 7.8 ms on the bar and 0.48 ms on the Armadillo's control mesh, against 14.7 and 0.84 ms for the PCG local step and 1.2 and 0.08 ms for three sweeps of the linearized relaxation. Since the results are the same, we keep the linearized local step.

The handle cells. The exact cells do not make large twists converge faster: with three exact sweeps per local step the 180° twist of the bar still takes about 2,500 iterations to settle (Figure 8). As Section 5.9 explains, the slow part is not the linearization but the treatment of the handle cells, and the



(A) $e = 0.2$ (B) $e = 0$ (C) At once



(D) Twist by height

FIGURE 7. A 270° twist of the bar (7,946 vertices), with the closed-form local step. (a), (b) The top handle is turned gradually, by 3° per step with 20 iterations per step, then held for 1,000 iterations, with feedback ($e = 0.2$) and without it ($e = 0$). (c) The handle is set to 270° at once, which places its vertices where −90° would; with the adaptive rule the handle's quaternion selects the 270° branch. (d) The mean rotation angle about the bar's axis of the control cells in each tenth of the bar's height.

measurements bear out each step of that explanation. Right after the handle of the bar turns by 180°, half of its 1,898 cells (949) have a pull angle above 90°, and the largest is 180°: their own fit and the mean of their neighbors are orthogonal quaternions. Their stiffness ratios r_2 and r_4 of Proposition 5.12 lie between 0.985 and 0.999, so the cell equation can only move them by a tiny step towards the handle, and at 180° by none at all. At the fixed point, by contrast, the largest pull angle of a handle cell is 18° on the twisted bar, 26° on the twisted arc, 10° on the 90° arc and 27° on the Armadillo, far below $\theta_{\max}$, which is what Proposition 5.13 needs.

To measure what the adaptive rule (5.17), which our method uses, contributes, we compare it with the cell equation (5.8) alone, on four scenes set at once and solved with the PCG local step: the twisted bar, the bar bent into a 180° arc and twisted by 180°, the 90° arc and the Armadillo (Table 5 and Figure 8). We count the iterations until the SR-ARAP energy stays within

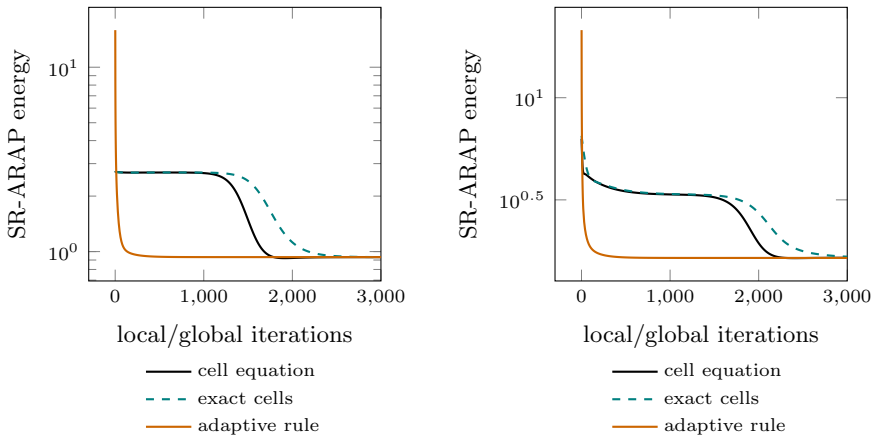
TABLE 4. The linearized cell against the exact cell on the unit sphere, at the same state. The multiplier gap is $|\lambda^* + \alpha A|/\alpha A$; the angle is between the normalized linearized rotation and the exact one; the pull angle is the largest angle, as a rotation, between a cell's own fit v_1 and the pull h_i of its neighbors; the steps are the mean Newton steps for the smallest eigenvalue and for the multiplier.

Scene	Cells	Count	Multiplier gap		Angle (deg)		Pull	Steps
			median	max	median	max	max	
Bar, arc 90°	free	6,048	$1.9 \cdot 10^{-4}$	$1.4 \cdot 10^{-3}$	$2 \cdot 10^{-5}$	$4 \cdot 10^{-4}$	42°	4.7 + 3.0
Bar, twist 180°	free	6,048	$7 \cdot 10^{-7}$	$5.6 \cdot 10^{-3}$	0	$9 \cdot 10^{-4}$	37°	3.8 + 2.3
Bar, twist 180°	handle, 1st	1,898	$3.5 \cdot 10^{-3}$	$1.5 \cdot 10^{-2}$	0	$9 \cdot 10^{-4}$	180°	1.3 + 1.9
Armadillo, control	free	316	$5.1 \cdot 10^{-5}$	$3.0 \cdot 10^{-2}$	$2 \cdot 10^{-6}$	0.15	38°	3.8 + 2.4

TABLE 5. Local/global iterations, and time, until the SR-ARAP energy stays within 1% of its final value, with the cell equation (5.8) alone for the handle cells and with the adaptive rule (5.17) of our method, $\theta_{\max} = 90°$; and the final energy of both, after 3,000 iterations (300 on the Armadillo).

Scene	Cell equation		Adaptive rule		Final energy	
	iterations	time (s)	iterations	time (s)	cell eq.	adaptive
Bar, twist 180°	2,030	30.8	390	6.2	0.9333	0.9334
Bar, arc + twist 180°	2,230	34.0	420	6.7	1.6471	1.6438
Bar, arc 90°	700	10.6	550	8.6	0.2233	0.2233
Armadillo, control	50	0.05	50	0.05	0.8111	0.8111

1% of its final value. On the twisted bar the adaptive rule needs 390 iterations instead of 2,030, and on the twisted arc 420 instead of 2,230: five times fewer. On the 90° arc, whose handle turns by 90° and whose handle cells therefore start near the threshold (377 of them above it), the rule acts during the first iterations and saves a fifth of them. On the Armadillo no handle cell ever exceeds 45°, the rule never departs from the cell equation, and the results are identical. In every case the two rules reach the same fixed point, as Proposition 5.13 predicts: the final energies agree to four digits, except on the twisted arc, where after 3,000 iterations the cell equation alone is still 0.2% above, and still decreasing. Figure 9 shows what the user sees: after 100 iterations the twisted bar is still pinched under the handle with the cell equation alone, and evenly twisted with the adaptive rule; after 300, the twisted arc is still flattened, and round with the adaptive rule. The threshold hardly matters: for $\theta_{\max}$ between 45° and 150° the twisted bar settles after the same 383 iterations, and the 90° arc after 358 to 691, always at the same energy. The rule costs one smallest-eigenvector computation per handle cell, which adds 0.6 ms to the 14.7 ms of a PCG local step on the bar.



(A) Bar, twist 180° (B) Bar, arc 180° and twist 180°

FIGURE 8. SR-ARAP energy per local/global iteration for large rotations set at once: the PCG local step with the cell equation (5.8) for the handle cells, three Gauss–Seidel sweeps of exact cells on the unit sphere (Section 5.8), and the PCG local step with the adaptive rule (5.17) for the handle cells.



(A) (B) (C) (D)

FIGURE 9. Large rotations set at once, with the cell equation and with the adaptive rule for the handle cells. (a), (b) The bar twisted by 180°, after 100 iterations: with the cell equation the handle cells stay at rest and the bar is pinched under the handle; with the adaptive rule the twist is already spread along the bar. (c), (d) The bar bent into a 180° arc and twisted by 180°, after 300 iterations.

9.8. Gallery

Figure 10 shows the bar benchmark twisted by 180°, bent into circular arcs of 90° and 180°, and twisted by 180° while following each arc. In the 180° arc the two ends of the bar meet the ground side by side, and the cross-section

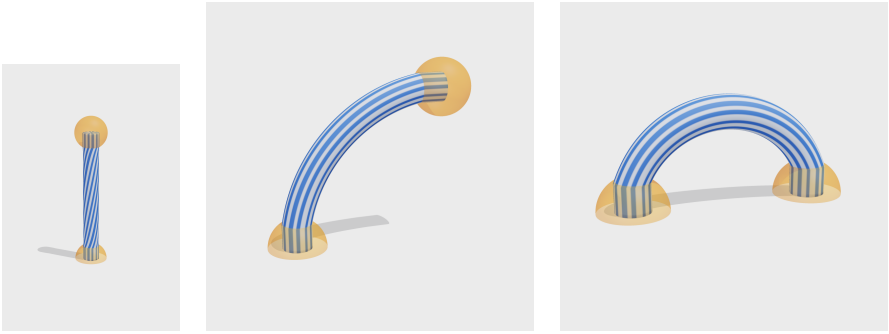
stays round along the whole arc. The stripes show that the twist is distributed evenly along the bar, as the smooth-rotation term intends: the rotation angle of the cells grows linearly with height, from 11° at the bottom to 162° just below the top handle. Large rotations need more local/global iterations than small ones: with the adaptive rule the 180° twist spreads along the bar in about 400 iterations, while with the cell equation alone for the handle cells it would take about 2,000, and after 600 it would still be concentrated under the top handle (Section 9.7). Figure 11 compares the 180° arc with the shape the original SR-ARAP local step converges to; they differ by at most 1.6% of the bar length, around the top of the arc. Figure 12 deforms the cactus with two handles. Bending the trunk along a 120° arc folds it over until its top points sideways, with the arms following rigidly; moving the top handle by two thirds of the trunk's length without rotating it bends the trunk into an S-curve, and the top keeps its orientation.

10. Conclusion

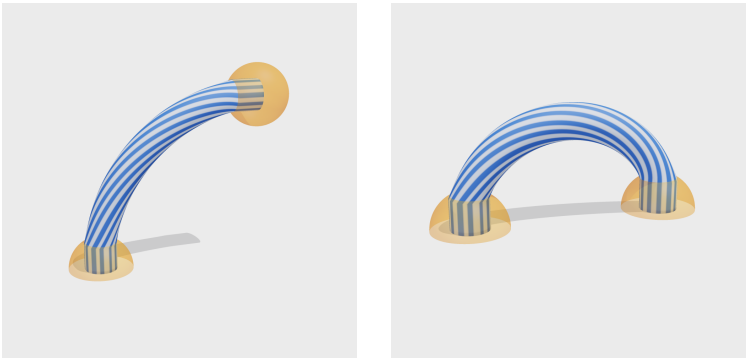
We have shown that the local step of SR-ARAP, when rotations are linearized as quaternions, can be solved exactly rather than by relaxation. Scaling the rows of the coupled per-cell equations makes them a single sparse, symmetric positive definite system, and that system is the exact stationarity condition of a quadratic energy, $\tilde{E}$, which differs from the linearized SR-ARAP energy by a valence weighting of the membrane term and an explicit proximal feedback term. The closed form is therefore not an approximation of relaxation: it is the minimizer that relaxation converges to. The same reduction makes every Poisson solve of the method symmetric positive definite, so the whole pipeline runs on sparse Cholesky factorizations without least-squares formulations.

The rotation system is also well conditioned. The feedback term bounds its smallest eigenvalue by the feedback weight, and its position-dependent part is small, of order $1/n$ relative to the rest; preconditioning with a Cholesky factorization of the position-independent part, factored once per set of constraints, bounds the condition number by $1 + \mu/e$, a constant that shrinks as the mesh is refined. Conjugate gradients with this preconditioner reach the exact rotations, to 10^{-12} rad, in 5 to 20 iterations, each costing a sparse product and one back-substitution. The exact local step therefore costs nearly linear time: on the largest tube it is 49 times faster than a direct factorization and as fast as a few relaxation sweeps, which are not exact.

The experiments support three further conclusions. First, exactness matters at the iteration budgets used in practice: two or three relaxation sweeps leave rotation errors of several degrees, and limited sweeps slow the decrease of the energy. Second, the converged shapes are of high quality: compared with the original SR-ARAP local step they have 5–12% lower ARAP distortion and smoother rotations, and they reach the distortion at which SR-ARAP stops 2.1 times sooner on a strongly bent bar and 4.3 times sooner on the Armadillo. Third, in the multiresolution setting the exact local step is a



(A) Twist 180° (B) Arc 90° (C) Arc 180°



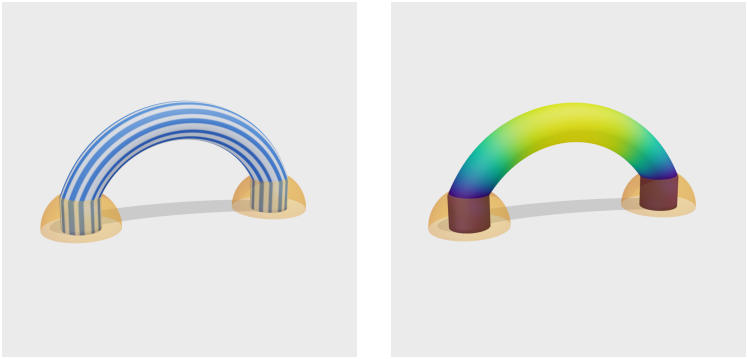
(D) Arc 90°, twist 180° (E) Arc 180°, twist 180°

FIGURE 10. The bar benchmark (7,946 vertices) after 2,000 local/global iterations with the closed-form local step. The bottom handle is fixed. The top handle is twisted about the bar's axis, moved along a circular arc while turning by the arc angle, or both.

small part of an interactive frame on a 173,000-vertex model, 0.9 ms on its 502-vertex control mesh.

The feedback term is part of the contribution. It gives temporal coherence while editing, favors the branch that large rotations are following, and bounds the smallest eigenvalue of the rotation system for any mesh and constraint set, while changing the converged shape by only a fraction of a percent. Its cost is a slower outer loop for very large rotations: on the bent bar the original SR-ARAP approaches its final shape in fewer local/global iterations, although it stops at a higher distortion, and our method is faster in wall-clock time only because each of its iterations is cheaper.

Finally, the linearization is a frozen Lagrange multiplier. Solving each cell exactly on the unit sphere, through its secular equation, shows that the true multiplier exceeds the frozen one by the cell's distortion and by the disagreement of its neighbors, which in our experiments keeps the linearized



(A) Closed form (B) SR-ARAP, 3 sweeps

FIGURE 11. The 180° arc with the closed-form local step and with the original SR-ARAP local step, both after 2,000 iterations. The colors show the distance between the two shapes.



(A) Rest (B) Arc 120° (C) Moved only

FIGURE 12. The cactus (5,261 vertices, 502-vertex control mesh) with its base fixed. (b) The trunk bent along a 120° arc: the top handle turns by 120° and moves to the end of the arc. (c) The top handle moved by two thirds of the trunk's length, sideways and down, without rotating it.

rotations within 0.15° of the exact ones. The same analysis locates the slow convergence of large twists, which the exact cells do not cure either: a handle turned by 180° has a rotation orthogonal, as a quaternion, to those of its neighbors, and the smoothing of the handle cells keeps them at rest. Our method therefore lets those cells take their own rotation in that case, and only then: the adaptive rule, which provably leaves the fixed points unchanged and reaches the same shapes five times sooner on the twisted bar and on the

twisted arc. An extension of the derivation to volumetric meshes is a natural direction for future work.

References

[1] I. Chao, U. Pinkall, P. Sanan, and P. Schröder. A simple geometric model for elastic deformations. In *ACM SIGGRAPH 2010 Papers*, SIGGRAPH '10, New York, NY, USA, 2010. Association for Computing Machinery.

[2] S.-Y. Chen, L. Gao, Y.-K. Lai, and S. Xia. Rigidity controllable as-rigid-as-possible shape deformation. *Graphical Models*, 91:13 – 21, 2017.

[3] W. Gander, G. H. Golub, and U. von Matt. A constrained eigenvalue problem. *Linear Algebra and its Applications*, 114–115:815–839, 1989.

[4] T. Jiang, K. Qian, S. Liu, J. Wang, X. Yang, and J. Zhang. Consistent as-similar-as-possible non-isometric surface registration. *The Visual Computer*, 33:891–901, 2017.

[5] Y. Le Vaou, J.-C. Léon, S. Hahmann, S. Masfrand, and M. Mika. As-stiff-as-needed surface deformationcombining arap energy with an anisotropic material. *Computer-Aided Design*, 121:102803, 2020.

[6] Z. Levi and C. Gotsman. Smooth rotation enhanced as-rigid-as-possible mesh animation. *IEEE transactions on visualization and computer graphics*, 21(2):264—277, February 2015.

[7] M. C. Lopez Belon. Applications of conformal geometric algebra in mesh deformation. In *Graphics, Patterns and Images (SIBGRAPI), 2013 26th SIBGRAPI - Conference on*, pages 39–46, Aug 2013.

[8] M. C. Lopez Belon. Project on github. Available at https://github.com/mauriciocele/arap-sr-linearized, 2020.

[9] J. J. Moré and D. C. Sorensen. Computing a trust region step. *SIAM Journal on Scientific and Statistical Computing*, 4(3):553–572, 1983.

[10] A. Morsucci, M. Centin, and A. Signoroni. Fast Centroidal Deformation for Large Mesh Models. In M. Livesu, G. Pintore, and A. Signoroni, editors, *Smart Tools and Apps for Graphics - Eurographics Italian Chapter Conference*. The Eurographics Association, 2018.

[11] U. Pinkall and K. Polthier. Computing discrete minimal surfaces and their conjugates. *Experimental Mathematics*, 2:15–36, 1993.

[12] O. Sorkine and M. Alexa. As-rigid-as-possible surface modeling. In *Proceedings of the fifth Eurographics symposium on Geometry processing*, SGP '07, pages 109–116, Aire-la-Ville, Switzerland, Switzerland, 2007. Eurographics Association.

[13] J. Wu, M. Lopez, M. Liu, and Y. Zhu. Linear geometric algebra rotor estimator for efficient mesh deformation. *IET Cyber-systems and Robotics*, 2(2):88–95, 2020.

[14] R. Zayer, C. Rössl, Z. Karni, and H. peter Seidel. Harmonic guidance for surface deformation. In *In Proc. of Eurographics 05*, pages 601–609, 2005.

Mauricio Cele Lopez Belon
Madrid, Spain
e-mail: mclopez@outlook.com